INTELLIGENT ATTENDANCE CONTROL AND STUDENT PROGRESS MONITORING PLATFORM

EASWARI ERP Portal

PROJECT REPORT

Submitted by

REHAN J (310625104288)

SAKTHISARAN M (310625104313)

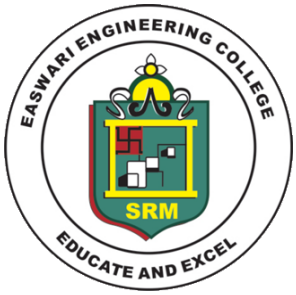
SANJAY N (310620104322)

In partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING



EASWARI ENGINEERING COLLEGE, RAMAPURAM

(Autonomous Institution)

affiliated to

ANNA UNIVERSITY: CHENNAI 600025

October 2026

EASWARI ENGINEERING COLLEGE, CHENNAI

(AUTONOMOUS INSTITUTION)

AFFILIATED TO ANNA UNIVERSITY, CHENNAI 600025

BONAFIDE CERTIFICATE

Certified that this project report titled “Intelligent Attendance Control and Student Progress Monitoring Platform – Easwari ERP Portal” is the bonafide work of “Rehan J (310625104288), Sakthisaran M (310625104313) and Sanjay N (310620104322)” who carried out the project work under my supervision during the academic year 2026–2027.

The work presented in this report has been carried out by the students and has not been submitted, either in part or in full, for the award of any other degree or diploma.

SIGNATURE
Dr. S. CLOUDIN
HEAD OF THE DEPARTMENT
Dept. of CSE
Easwari Engineering College
Ramapuram, Chennai-600089

SIGNATURE
MS. BHAGYALAKSHMI A
SUPERVISOR
Dept. of CSE
Easwari Engineering College
Ramapuram, Chennai-600089

ACKNOWLEDGEMENT

We thank the Almighty God for His abundant blessings and guidance throughout the successful completion of this project.

We express our sincere thanks and deep sense of gratitude to our Founder Chairman, Dr. T. R. Parivendhar, and our beloved Chairman, Dr. R. Shivakumar, M.D., Ph.D., for their constant support, encouragement, and inspiration which motivated us to successfully complete this project.

We would like to record our respectful thanks to our Principal, Dr. P. Deiva Sundari, M.E., Ph.D., and our Head of the Department, Dr. S. Cloudin, for their valuable guidance, encouragement, and continuous support throughout the project.

We owe our profound gratitude to our project supervisor, Ms. Bhagyalakshmi A, Department of Computer Science and Engineering, Easwari Engineering College, for her valuable suggestions, constant encouragement, and keen interest in our work. Her guidance at every stage helped us to complete this project in its present form.

We also sincerely thank our project coordinator, Mrs. S. Sushmitha, for her continuous assistance and support during the course of this work.

We extend our heartfelt thanks to all the teaching and non-teaching staff members of the Department of Computer Science and Engineering for their cooperation and encouragement which greatly facilitated the smooth completion of this project.

Finally, we express our sincere thanks to our parents, friends, and all those who directly or indirectly helped us in completing this project successfully.

REHAN J (310625104288)
SAKTHISARAN M (310625104313)
SANJAY N (310620104322)

ABSTRACT

The Easwari ERP Portal is a web-based campus platform for attendance control and student progress monitoring. In many colleges attendance and internal marks are kept in paper registers and spreadsheets that different people update at different times, so the figures that a student, a coordinator and an examination cell see often disagree, a class can be marked by a teacher who does not handle it, and short attendance is discovered too late to be corrected. The proposed system keeps one authoritative record of each fact and controls who may change it. It provides three role-specific portals, namely Student, Faculty and Administrator, behind one sign-in. The administrator provisions accounts with random temporary passwords that must be changed at the first login, and students may later create a six-digit quick code for faster sign-in, which is locked after five wrong attempts.

Attendance is stored once for each student, subject offering, date and period, under a database constraint that makes double marking impossible, and the period, day and subject views are all derived from that record. The class coordinator can mark and correct every period of the class, a subject teacher can change only his or her own subject, and the administrator has oversight of every class; the server enforces these rules for every request. Internal marks and laboratory components are entered with a maximum for each assessment and are visible to students only after publication. Students see their attendance register, subject-wise eligibility against the 75% requirement, internal marks, practicals, university results and CGPA, together with an itemised fee portal with simulated payment and receipts, campus service requests, leave and on-duty applications with proof upload, placement matches with server-checked eligibility, and a Career Hub with a PDF resume analyzer, GitHub analyzer, skill-gap analysis, courses, certifications, project blueprints and a study assistant.

The system is implemented with Java 17 and Spring Boot 3.2.5 for the API, a relational model on an embedded H2 file database that moves to PostgreSQL by configuration, JSON Web Tokens with BCrypt password hashing for security, React 18 with Vite for the interface, and a separate FastAPI service that computes an explainable academic-risk score and a placement-readiness score. A trial institution of five departments, ten classes, 59 students and 25 staff members was seeded and exercised. Sixty-eight recorded test cases and eleven unit tests were run against the system; the rules for authority, validation, fees, leave and eligibility held in all but one case, and the one defect found is reported with its fix. The result is a modular foundation for a more accurate, accountable and student-friendly academic administration process.

KEYWORDS: Attendance Management, Student Progress Monitoring, College ERP, Role-Based Access Control, Spring Boot, React, JWT, Academic Risk Score, Career Hub, FastAPI.

TABLE OF CONTENTS

BONAFIDE CERTIFICATE **2**

ACKNOWLEDGEMENT **3**

ABSTRACT **4**

LIST OF TABLES **8**

LIST OF FIGURES **9**

LIST OF ABBREVIATIONS **11**

CHAPTER 1 – INTRODUCTION **12**

1.1 INTRODUCTION 12

1.2 BACKGROUND 12

1.3 AIM 12

1.4 OBJECTIVES 13

1.5 SCOPE OF THE PROJECT 13

1.6 EXISTING SYSTEM 14

1.7 PROPOSED SOLUTION 14

1.8 ADVANTAGES 14

1.9 ORGANIZATION OF THE REPORT 14

CHAPTER 2 – RELATED WORK **15**

2.1 INTRODUCTION 15

2.2 RELATED WORK 1: QR CODE-BASED ATTENDANCE SYSTEM USING WEB TECHNOLOGY 15

2.3 RELATED WORK 2: USING LEARNING ANALYTICS TO DEVELOP AN EARLY-WARNING SYSTEM FOR AT-RISK STUDENTS 15

2.4 RELATED WORK 3: CONTRASTING PREDICTION METHODS FOR EARLY WARNING SYSTEMS AT UNDERGRADUATE LEVEL 16

2.5 COMPARATIVE ANALYSIS 16

2.6 RESEARCH GAP 17

2.7 SUMMARY 17

CHAPTER 3 – SYSTEM ANALYSIS **18**

3.1 PROBLEM DEFINITION 18

3.2 PROPOSED SYSTEM 18

3.3 SOFTWARE COMPONENTS 19

3.4 HARDWARE COMPONENTS 19

3.5 FUNCTIONAL REQUIREMENTS 20

3.6 NON-FUNCTIONAL REQUIREMENTS 20

3.7 FEASIBILITY STUDY 21

CHAPTER 4 – SYSTEM DESIGN **22**

4.1 DESIGN OF THE PROJECT 22

4.2 SYSTEM ARCHITECTURE DIAGRAM 23

4.3 MODULES 24

CHAPTER 5 – IMPLEMENTATION **27**

5.1 DEVELOPMENT ENVIRONMENT 27

5.2 STEP-BY-STEP IMPLEMENTATION 27

5.3 FRONTEND COMPONENT STRUCTURE 27

5.4 SPRING BOOT SERVER AND API ROUTES 28
5.5 DATABASE MODEL AND CONSTRAINTS 29
5.6 AUTHENTICATION AND AUTHORIZATION 29
5.7 ATTENDANCE AUTHORITY AND ELIGIBILITY LOGIC 30
5.8 MARKS ENTRY AND PUBLISH GATE 30
5.9 ACADEMIC RISK SCORE 30
5.10 FEE PAYMENT AND RECEIPT LOGIC 31
5.11 LEAVE AND ON-DUTY WORKFLOW 31
5.12 ERROR HANDLING AND VALIDATION 31
5.13 DEPLOYMENT CONSIDERATIONS 31

CHAPTER 6 – SYSTEM TESTING 32
6.1 TESTING OBJECTIVE 32
6.2 TEST CASE FORMAT 32
6.3 TESTING TYPES 37

CHAPTER 7 – OUTPUT AND EXPLANATION 38
7.1 WELCOME AND ROLE SELECTION 38
7.2 STUDENT SIGN-IN 39
7.3 QUICK-CODE SIGN-IN 39
7.4 ADMINISTRATOR DASHBOARD 40
7.5 ACCESS MANAGEMENT 40
7.6 CREATE AN ACCOUNT 41
7.7 RISK AND ALERTS 41
7.8 DEPARTMENTS AND CLASSES 42
7.9 DEPARTMENT DETAIL AND HOD EDIT 42
7.10 COURSES AND TIMETABLE 43
7.11 LEAVE REQUESTS (ADMINISTRATION) 43
7.12 SERVICE QUEUE 44
7.13 AUDIT TRAIL 44
7.14 FACULTY DASHBOARD 45
7.15 ATTENDANCE ENTRY BY THE CLASS COORDINATOR 45
7.16 ATTENDANCE ENTRY BY A SUBJECT TEACHER 46
7.17 MARKS ENTRY 46
7.18 LEAVE REVIEW 47
7.19 STUDENT DASHBOARD 47
7.20 ATTENDANCE REGISTER 48
7.21 INTERNAL MARKS 48
7.22 ASSESSMENTS 49
7.23 PRACTICALS 49
7.24 UNIVERSITY EXAMINATION RESULTS 50
7.25 FEE PORTAL 50
7.26 CAMPUS SERVICES 51
7.27 LEAVE AND ON-DUTY APPLICATION 51
7.28 LEARNING PATH 52
7.29 PLACEMENT 52
7.30 INTERNSHIPS 53
7.31 ROADMAP 53
7.32 SETTINGS AND QUICK CODE 54
7.33 CAREER HUB OVERVIEW 54
7.34 RESUME ANALYZER 55

7.35 GITHUB ANALYZER 55
7.36 SKILL GAP ANALYSIS 56
7.37 COURSES 56
7.38 CERTIFICATIONS AND ACHIEVEMENTS 57
7.39 PROJECT BLUEPRINTS 57
7.40 STUDY ASSISTANT 58

CHAPTER 8 – RESULTS AND DISCUSSIONS 59
8.1 RESULTS 59
8.2 DISCUSSION 59
8.3 LIMITATIONS 59
8.4 EVALUATION METRICS TO RECORD 60

CHAPTER 9 – CONCLUSION AND FUTURE ENHANCEMENT 61
9.1 CONCLUSION 61
9.2 FUTURE ENHANCEMENTS 61
9.3 RECOMMENDATIONS FOR FUTURE DEVELOPERS 61

CHAPTER 10 – APPENDIX 63
10.1 ATTENDANCE UNIQUENESS CONSTRAINT 63
10.2 ROLE AUTHORIZATION PATTERN 63
10.3 RISK SCORE COMPARISON 63
10.4 KEY VIVA QUESTIONS 64

CHAPTER 11 – REFERENCES 65

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
2.1	Comparison of related work and the proposed system	16
3.1	Software components and libraries	19
3.2	Hardware requirements	19
4.1	Main data entities	22
5.1	Development environment	27
5.2	Main front-end files	28
5.3	Selected API routes	28
5.4	Weights used by the academic-risk score	31
6.1	Test documentation – Valid and invalid login	32
6.2	Test documentation – Account provisioning and first login	32
6.3	Test documentation – Password change and quick code	33
6.4	Test documentation – Quick-code lockout	33
6.5	Test documentation – Who may mark attendance	33
6.6	Test documentation – Attendance validation and duplicate marking	33
6.7	Test documentation – Marks entry rules	34
6.8	Test documentation – Publish marks	34
6.9	Test documentation – Fee statement and payment	34
6.10	Test documentation – Leave and on-duty workflow	35
6.11	Test documentation – Campus service requests	35
6.12	Test documentation – Placement eligibility	35
6.13	Test documentation – Unauthorized access	35
6.14	Test documentation – Malformed requests	36
6.15	Test documentation – Account access and password reset	36
6.16	Test documentation – Administrative actions and audit trail	36
6.17	Test documentation – Scoring service availability	36
6.18	Test documentation – Database restore drill	37
8.1	Evaluation metrics recorded and to be recorded	60
10.1	Risk score for three profiles	64

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
4.1	Layered architecture of Easwari ERP Portal	23
4.2	Core data model centred on the subject offering	24
4.3	Authority check performed for every attendance request	25
7.1	Welcome and role selection	38
7.2	Student sign-in	39
7.3	Quick-code sign-in	39
7.4	Administrator dashboard	40
7.5	Access management	40
7.6	Create an account	41
7.7	Risk and alerts	41
7.8	Departments and classes	42
7.9	Department detail and HOD edit	42
7.10	Courses and timetable	43
7.11	Leave requests (administration)	43
7.12	Service queue	44
7.13	Audit trail	44
7.14	Faculty dashboard	45
7.15	Attendance entry by the class coordinator	45
7.16	Attendance entry by a subject teacher	46
7.17	Marks entry	46
7.18	Leave review	47
7.19	Student dashboard	47
7.20	Attendance register	48
7.21	Internal marks	48
7.22	Assessments	49
7.23	Practicals	49
7.24	University examination results	50
7.25	Fee portal	50
7.26	Campus services	51
7.27	Leave and on-duty application	51
7.28	Learning path	52
7.29	Placement	52

FIGURE NO.	TITLE	PAGE NO.
7.30	Internships	53
7.31	Roadmap	53
7.32	Settings and quick code	54
7.33	Career Hub overview	54
7.34	Resume analyzer	55
7.35	GitHub analyzer	55
7.36	Skill gap analysis	56
7.37	Courses	56
7.38	Certifications and achievements	57
7.39	Project blueprints	57
7.40	Study assistant	58

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
API	Application Programming Interface
BCrypt	Password hashing function based on the Blowfish cipher
CGPA	Cumulative Grade Point Average
CORS	Cross-Origin Resource Sharing
CSS	Cascading Style Sheets
ERP	Enterprise Resource Planning
HOD	Head of the Department
HTTP	Hyper Text Transfer Protocol
HTTPS	Hyper Text Transfer Protocol Secure
IA	Internal Assessment
JPA	Java Persistence API
JSON	JavaScript Object Notation
JWT	JSON Web Token
OD	On Duty
ORM	Object Relational Mapping
PDF	Portable Document Format
REST	Representational State Transfer
SQL	Structured Query Language
UI	User Interface
UPI	Unified Payments Interface
URL	Uniform Resource Locator
UX	User Experience

CHAPTER 1 – INTRODUCTION

1.1 INTRODUCTION

Attendance and internal assessment are the two records on which almost every academic decision in an engineering college depends. A student is allowed into the end-semester examination only if the attendance requirement is met, the internal marks contribute directly to the final grade, and the same two records are used by mentors, class coordinators and heads of department to decide whom to counsel. In many institutions these records are still maintained in paper registers and spreadsheets that are updated at different times by different people, so the percentage that a student sees, the percentage that a coordinator calculates and the percentage that finally reaches the examination cell often do not agree.

The Easwari ERP Portal is a web-based campus platform that places attendance control, internal assessment, examination results, fee collection, campus service requests, leave and on-duty approvals, and placement preparation into one system. It is built around three role-specific portals, namely the Student Portal, the Faculty Portal and the Administrator Portal, and a single sign-in that decides which portal a person sees. The core of the system is a period-level attendance record that is stored once and read in three ways: per period, per day and per subject. Everything else on a student’s dashboard, such as the percentage, the subject-wise table, the list of subjects below the eligibility line and the risk indicator, is calculated from that single record.

1.2 BACKGROUND

Most college offices follow a familiar routine. Each faculty member marks attendance in a register at the end of the hour, a class coordinator consolidates the registers at the end of the month, and the figures are typed into a spreadsheet for display on the notice board. Internal marks follow the same path from the answer script to a mark list and from the mark list to a spreadsheet. A fee receipt is a paper slip, a bonafide or hall-ticket request is a handwritten form, and an on-duty letter travels from the student to the coordinator and the head of the department by hand.

This routine has predictable weaknesses. A period can be marked twice or not at all. A subject teacher can alter the attendance of a class that he or she does not teach. A student who is short of attendance learns about it only when the hall-ticket list is displayed, which is too late to recover. The management sees an average for a department but cannot tell which five students are responsible for it. A centralized, role-aware system with a database that enforces its own rules removes these weaknesses at the source rather than relying on staff to follow a procedure.

1.3 AIM

The aim of the project is to design and implement a modular, role-based web platform that keeps one authoritative record of attendance and internal assessment, enforces who may change which

part of it, and turns the stored data into timely information for students, faculty and administrators.

1.4 OBJECTIVES

- Provide separate Student, Faculty and Administrator portals behind one secure sign-in, with temporary passwords for new accounts and a compulsory change at first login.
- Record attendance per period with a database constraint that makes double-marking impossible, and offer both a single-period path and a whole-day path for faculty.
- Allow the class coordinator to mark and correct every period of the class, restrict a subject teacher to his or her own subject, and give the administrator oversight of every class.
- Capture internal assessments and laboratory components with per-assessment maximum marks, and release marks to students only when they are published.
- Show every student an attendance register, subject-wise percentage, eligibility status, internal marks, practical marks, university results and CGPA.
- Provide a fee portal with itemised statements, simulated payment and printable receipts, and a campus-services queue for requests such as hostel, transport, ID card, exam cell and library.
- Support leave and on-duty applications with proof upload, review by the staff of the class, and automatic OD entries in the attendance register on approval.
- Give students a Career Hub with a PDF resume analyzer, GitHub analyzer, skill-gap analysis, course list, certification tracker, project blueprints and a study assistant.
- Compute an explainable academic-risk score and placement-readiness score in a separate scoring service so that mentors can act early.
- Maintain an audit trail of privileged administrative actions and keep the whole design portable from a file database to PostgreSQL.

1.5 SCOPE OF THE PROJECT

The scope covers the React front end, the Spring Boot REST API, the relational data model, the FastAPI scoring service and the seeding of a complete institution consisting of five departments, ten classes, about sixty students and twenty-five staff members. It includes authentication with JSON Web Tokens, the quick-code sign-in for students, the attendance, marks, fee, leave and service workflows, Excel import and export, and the Career Hub tools.

The scope does not include a live payment gateway, biometric or RFID attendance capture, or integration with the University’s own examination system. Payment is simulated inside the application and the university results are entered through the data seeder or by the administration. These boundaries are stated again in Chapter 8 so that the reader can judge the results against what was actually attempted.

1.6 EXISTING SYSTEM

In the existing manual or semi-digital process, attendance lives in registers and marks in spreadsheets. There is no single place where a student, a faculty member and an administrator see the same figure. Corrections are made by overwriting a cell, so there is no record of who changed what. Eligibility for the examination is calculated at the end of the semester, and campus requests are handled through paper forms and personal visits to the concerned office.

1.7 PROPOSED SOLUTION

The proposed solution stores each attendance mark as one row identified by student, subject offering, date and period, and derives every report from those rows. Access rules are enforced on the server for every request, so hiding a menu item in the browser is never the only protection. Students see their own data only; the identity of the student is read from the token and never from the address of the page. Faculty see the classes they teach, and the coordinator of a class sees and edits the whole class. The administrator creates accounts, departments, classes and timetables and can inspect every action in the audit trail.

1.8 ADVANTAGES

- One record of attendance and marks, read consistently by every screen.
- Server-side enforcement of who may mark attendance or enter marks for a class.
- Early visibility of subjects that have fallen below the 75% eligibility line.
- Temporary random passwords, forced first-login change and a lockout on repeated wrong quick codes.
- Fee statements, receipts, leave and service requests handled without paper.
- Explainable risk and readiness scores that list the factors behind every number.
- A configuration-only migration path from the embedded database to PostgreSQL.

1.9 ORGANIZATION OF THE REPORT

Chapter 1 introduces the project and its objectives. Chapter 2 reviews related published work and identifies the gap addressed here. Chapter 3 analyses the problem, the proposed system and the software and hardware requirements. Chapter 4 presents the architecture, the data model and the modules. Chapter 5 explains the implementation of the main algorithms and security features. Chapter 6 documents the test cases that were executed against the running system. Chapter 7 presents the screens of the implemented portals with explanations. Chapter 8 discusses the results and the limitations, Chapter 9 concludes and lists future enhancements, Chapter 10 gives short code listings and viva questions, and Chapter 11 lists the references.

CHAPTER 2 – RELATED WORK

2.1 INTRODUCTION

Published work relevant to this project falls into two groups: web-based attendance systems that replace the paper register, and learning-analytics studies that use attendance and assessment data to identify students who may fail. Three published studies were selected, one from the first group and two from the second, and each is compared with the Easwari ERP Portal.

2.2 RELATED WORK 1: QR CODE-BASED ATTENDANCE SYSTEM USING WEB TECHNOLOGY

Ramadhani and Satria (2025) describe a web-based student attendance system in which every student receives a unique QR code that is scanned with a camera-equipped device and saved to a central database. The system provides user login, QR code generation, attendance scanning and automatic report creation, and was developed through requirements analysis, design, implementation and testing. Functional testing reported that attendance is recorded and reports are produced in real time, reducing manual errors and speeding up processing [1].

The strength of this work is the removal of the paper register and the immediate availability of reports. Its description centres on the capture of attendance. It does not discuss which member of staff is entitled to change which class, how a mistaken mark is corrected, or how attendance is connected to internal marks, leave and eligibility. Relevance to the proposed project: it confirms that a central database with automatic reports is a sound replacement for registers. The Easwari ERP Portal adds role-based rules for who may mark and correct attendance, a unique constraint that prevents duplicate marks, and the link to leave, marks and eligibility.

2.3 RELATED WORK 2: USING LEARNING ANALYTICS TO DEVELOP AN EARLY-WARNING SYSTEM FOR AT-RISK STUDENTS

Akçapınar, Altun and Aşkar (2019) analysed the online learning interaction data of 76 second-year students in a Computer Hardware course to test whether end-of-term performance could be predicted early. They compared several classification algorithms and preprocessing methods. The k-nearest neighbours algorithm identified students who would be unsuccessful at the end of the term with about 89% accuracy, and data from the third week of the course already allowed unsuccessful students to be flagged with about 74% accuracy [2].

The study shows that early intervention is feasible when data is collected continuously. Its models, however, depend on interaction logs from an online learning platform and on a statistical classifier trained on a single course. A classroom-taught engineering programme does not generate such logs, and a classifier trained on 76 students is not something a college can rely on without validation. Relevance to the proposed project: the Easwari ERP Portal uses the data that every college already has, namely attendance and internal marks, and presents it through a

transparent weighted score rather than a trained model, so that a mentor can see exactly why a student has been flagged.

2.4 RELATED WORK 3: CONTRASTING PREDICTION METHODS FOR EARLY WARNING SYSTEMS AT UNDERGRADUATE LEVEL

Howard, Meehan and Parnell (2017) compared eight prediction methods, including Bayesian additive regression trees, random forests, gradient boosting and neural networks, for forecasting the final percentage grades of students in a large first-year statistics course. The models used engagement data from the learning-management system, continuous assessment marks, background information and cluster membership. Continuous assessment was the most important predictor, and the authors identified weeks five to six of the semester as the best balance between prediction accuracy and the time left for intervention. The best method predicted final grades by week six with a mean absolute error of about 6.5 percentage points [3].

The finding that continuous assessment is the strongest predictor supports the weight given to internal marks in the Easwari ERP risk score, and the timing result supports showing the score throughout the semester rather than only at its end. The study also reminds the reader that predictive tools support decisions and do not replace them. Relevance to the proposed project: the risk indicator in the Easwari ERP Portal is deliberately presented as a weighted and explainable score and not as a prediction.

2.5 COMPARATIVE ANALYSIS

Table 2.1: Comparison of related work and the proposed system

Feature / Concern	QR attendance (2025)	Early-warning, kNN (2019)	Early-warning, BART (2017)	Easwari ERP Portal
Attendance capture	QR scan to central database	Not used	Not used	Period-level marking by faculty with unique constraint
Who may change a record	Not described	Not applicable	Not applicable	Coordinator: whole class; subject teacher: own subject; administrator: oversight
Internal marks	Not covered	Course performance as target	Continuous assessment as predictor	Assessments and lab components with publish gate
Early identification	Reports only	kNN classifier on interaction logs	Eight methods compared	Transparent weighted risk score with listed factors
Student self-service	Report view	Not described	Not described	Attendance, marks, fees, leave, services, placement
Main gap to consider	No role rules or eligibility link	Needs LMS logs; single course	Needs LMS data; single course	Needs validation with real outcome data

2.6 RESEARCH GAP

The attendance system reviewed above captures attendance but does not define the authority to change it. The early-warning studies assume a learning-management system that most affiliated engineering colleges do not run. What a college actually needs is an integrated record that combines attendance, internal marks, leave, fees and services, that states clearly who may modify each part, and that produces an early signal from data the college already holds. The Easwari ERP Portal is designed to fill that gap.

2.7 SUMMARY

The literature shows a clear move from paper registers to central databases and from end-of-term results to early warning. The proposed system combines the two directions in a single platform for an affiliated engineering college and adds server-enforced authority rules. Its main task, taken up in the following chapters, is to demonstrate that these rules hold under test.

CHAPTER 3 – SYSTEM ANALYSIS

3.1 PROBLEM DEFINITION

Attendance, internal marks, fees and student requests are kept in separate places and updated by different people. As a result the same student can show different attendance on different lists, a class can be marked by a teacher who does not handle it, and short attendance is discovered too late to be corrected. The problem is to build one platform that keeps a single record of each fact, controls who may change it, and presents the right view to the right person.

3.2 PROPOSED SYSTEM

The proposed system is a responsive web application with a component-based React front end, a Spring Boot REST API and a relational database. There are three kinds of account. An administrator provisions departments, programmes, classes, subject offerings, timetables and user accounts. A faculty member sees the classes he or she teaches; if he or she is also the class coordinator, the whole class is open for attendance and marks. A student sees only his or her own record.

The central data object is the subject offering, which represents one subject taught to one class section by one member of staff. Timetable slots, attendance records and assessments hang from the offering, which is why a statement such as “Computer Networks attendance” has a precise meaning. Attendance is stored once for each combination of student, offering, date and period, and the faculty can submit a single period or a whole day. A separate scoring service receives a compact feature payload from the API and returns a risk score, a placement-readiness score, recommendations and assistant replies; the API continues to work when that service is switched off.

3.3 SOFTWARE COMPONENTS

Table 3.1: Software components and libraries

Component	Technology / Library	Purpose
Programming language (server)	Java 17	Application logic and API
Application framework	Spring Boot 3.2.5	REST controllers, dependency injection, validation
Persistence	Spring Data JPA with Hibernate	Object-relational mapping and repositories
Database	H2 (file mode); PostgreSQL by configuration	Relational storage with constraints
Security	Spring Security, JJWT, BCrypt	Authentication, role checks, password hashing
Spreadsheet support	Apache POI 5.2.5	Bulk user import and Excel exports
Build tool	Apache Maven	Compilation, testing and packaging
Front-end framework	React 18 with Vite 5	Component-based screens and fast builds
Charts and icons	Recharts, Lucide React	Dashboard charts and interface icons
PDF reading	pdf.js (pdfjs-dist)	Text extraction for the resume analyzer in the browser
Visual background	LiquidEther (WebGL)	Animated background on sign-in and Career Hub
Scoring service	Python 3, FastAPI, Uvicorn, NumPy	Risk score, readiness score, recommendations, assistant
Testing	JUnit 5, Spring Security Test	Unit tests of attendance, marks, faculty and password rules

3.4 HARDWARE COMPONENTS

Table 3.2: Hardware requirements

Component	Minimum	Recommended
Processor	Dual-core 2.0 GHz	Quad-core 2.5 GHz or higher
Memory (RAM)	4 GB	8 GB or more
Storage	2 GB free space	10 GB free space with backup disk
Display	1366 x 768	1920 x 1080
Network	Local network connection	Wired or Wi-Fi campus network with HTTPS
Client device	Any current browser on a laptop, tablet or phone	Chrome or Edge, latest version

The server runs comfortably on an ordinary laptop, as was done during development and testing. A browser that supports WebGL is needed for the animated background; on other browsers the interface remains fully usable.

3.5 FUNCTIONAL REQUIREMENTS

- FR1: Sign in with a username and password; students may additionally sign in with a six-digit quick code.
- FR2: Provision accounts individually or in bulk from a spreadsheet and issue a random temporary password that must be changed at first sign-in.
- FR3: Mark attendance for one period or for a whole day, with the statuses Present, Absent, On duty and Medical.
- FR4: Let the class coordinator mark and edit every period of the class and let a subject teacher edit only his or her own subject.
- FR5: Enter internal marks and laboratory components with a maximum mark per assessment, keep them as drafts and publish them to students.
- FR6: Show students their attendance, internal marks, assessments, practicals, university results and CGPA.
- FR7: Present an itemised fee statement, accept simulated payment and issue a receipt number.
- FR8: Accept leave and on-duty applications with proof, route them to the staff of the class and update attendance on approval.
- FR9: Raise and track campus service requests and let the administrator resolve them.
- FR10: Show placement matches with server-checked eligibility, internships, a roadmap and a learning path.
- FR11: Provide the Career Hub tools for resume, GitHub, skills, courses, certifications, projects and study material.
- FR12: Record privileged administrative actions in an audit trail.

3.6 NON-FUNCTIONAL REQUIREMENTS

- Security: BCrypt password hashing, signed tokens, role checks on every route, ownership checks in the services and a lockout after five wrong quick codes.
- Integrity: database constraints for the attendance slot, unique usernames and unique placement applications.
- Usability: a plain, consistent interface with clear page titles, status chips and empty states instead of blank screens.
- Portability: configuration-only change from the file database to PostgreSQL.
- Resilience: dashboards continue to render when the scoring service is unavailable.

- Maintainability: separate controller, service, repository and domain layers and unit tests for the rules that matter most.

3.7 FEASIBILITY STUDY

Technical feasibility is established by the fact that every component is open source, runs on a single laptop and was assembled into a working system during the project. Economic feasibility follows from the same fact: no licence, hosting or hardware cost is needed for the demonstration, and a college could host the system on one small server. Operational feasibility depends on the willingness of faculty to mark attendance in the portal; the single-period path, which pre-fills the roster with Present, was designed to make this quicker than a paper register.

CHAPTER 4 – SYSTEM DESIGN

4.1 DESIGN OF THE PROJECT

The design follows one principle: the Java service owns the data and the Python service owns the scoring models, and neither reaches into the other's storage. The scoring service holds no database connection. The API assembles a feature payload and posts it; the scoring service returns a score. Because of this boundary the two can be scaled or replaced independently, and the portals still render when the scoring service is switched off.

A second principle is that security is layered. Route guards map the address prefixes /api/admin, /api/faculty and /api/student to roles. Method-level guards sit on every controller. Services then check ownership: the student controller resolves the student from the token and never from a path parameter, and the attendance and marks services reject a staff member who neither teaches the offering nor coordinates the class. Finally the eligibility rules, such as the CGPA and attendance conditions of a placement drive, are checked again on the server when a student applies, so disabling a button in the browser achieves nothing.

The third principle concerns attendance. The requirement was to see attendance by period, by day and by subject. Storing three tables would require three things to be kept in sync, so the finest grain is stored once, as one row per student, offering, date and period, and the other views are aggregations. A unique constraint on those four columns turns a second submission of the same period into an update.

Table 4.1: Main data entities

Entity	Purpose	Notes
User	Login account for all three roles	Username is a roll number for students and an employee code for staff; mustChangePassword flag; quick-code hash and lockout fields
Department / Program	Organisation of the institution	Programme duration is a field, not a constant
ClassSection	A cohort, for example CSE II-E	Holds the class coordinator
Student / Faculty	Profiles linked to a user	Student carries cached CGPA and credits
Subject / SubjectOffering	Catalogue entry and its delivery	Offering = subject + section + teacher; lab flag for practicals
TimetableSlot	Weekly recurring period	Clash between slots is rejected with 409
AttendanceRecord	One mark per student, offering, date, period	Unique constraint uk_attendance_slot
Assessment / Mark	Internal assessment and its scores	Max marks per assessment; published flag; null means not entered
UniversityResult	Semester examination result	Grade, grade points and credits
FeeItem	A line on a student's fee statement	Receipt number and payment mode when paid
LeaveApplication	Leave or OD request	Proof file, reviewer, review note

Entity	Purpose	Notes
CmsRequest	Campus service ticket	Open, in progress, resolved, rejected
Company / PlacementApplication / Internship	Placement drives and openings	Eligibility stored as data
SkillScore	Aptitude and interview scores	Used by the readiness score
AuditLog	Privileged actions	Actor, action, target, time

4.2 SYSTEM ARCHITECTURE DIAGRAM

Figure 4.1 shows the layered architecture. The browser communicates with the application layer through JSON over HTTP with a bearer token. The application layer reaches the relational database through Spring Data and the scoring service through an HTTP client.

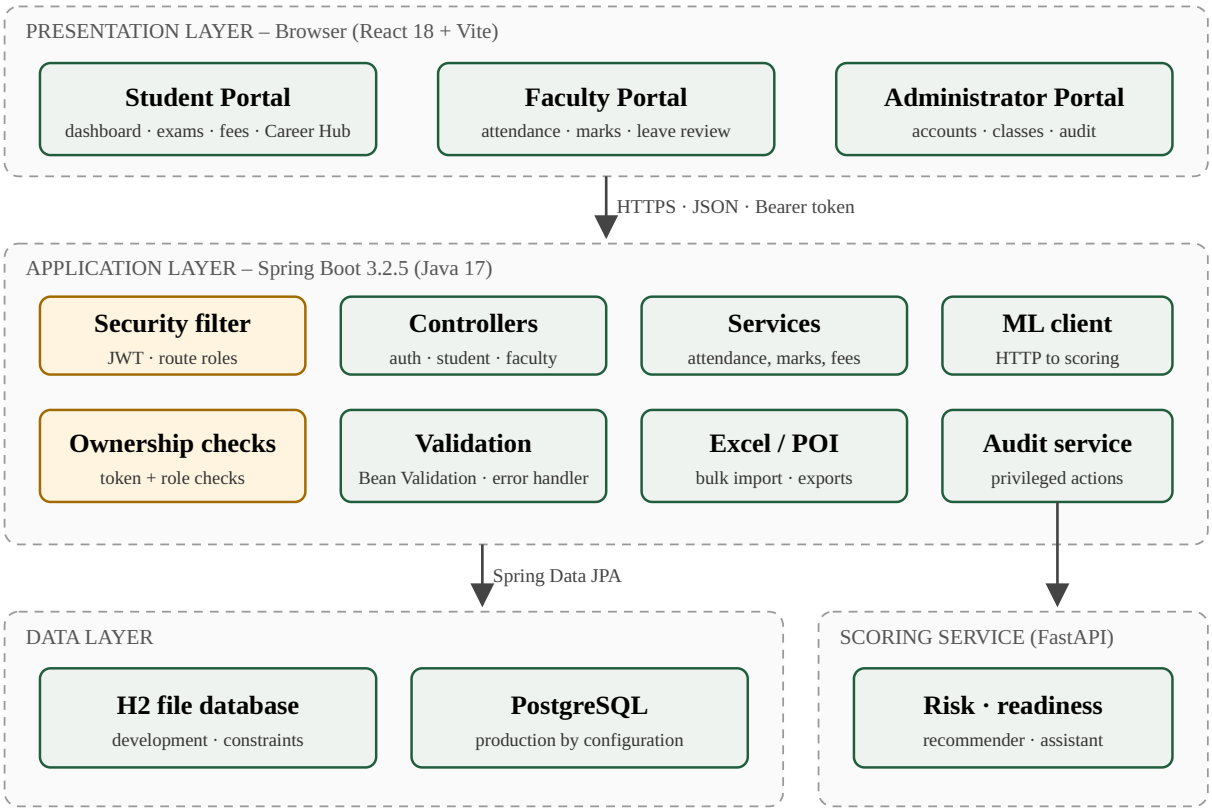


Figure 4.1: Layered architecture of Easwari ERP Portal

The data model in Figure 4.2 is organised around the subject offering. Attendance records and assessments belong to an offering, not directly to a subject, because attendance in a subject is meaningful only once the section and the teacher are known.

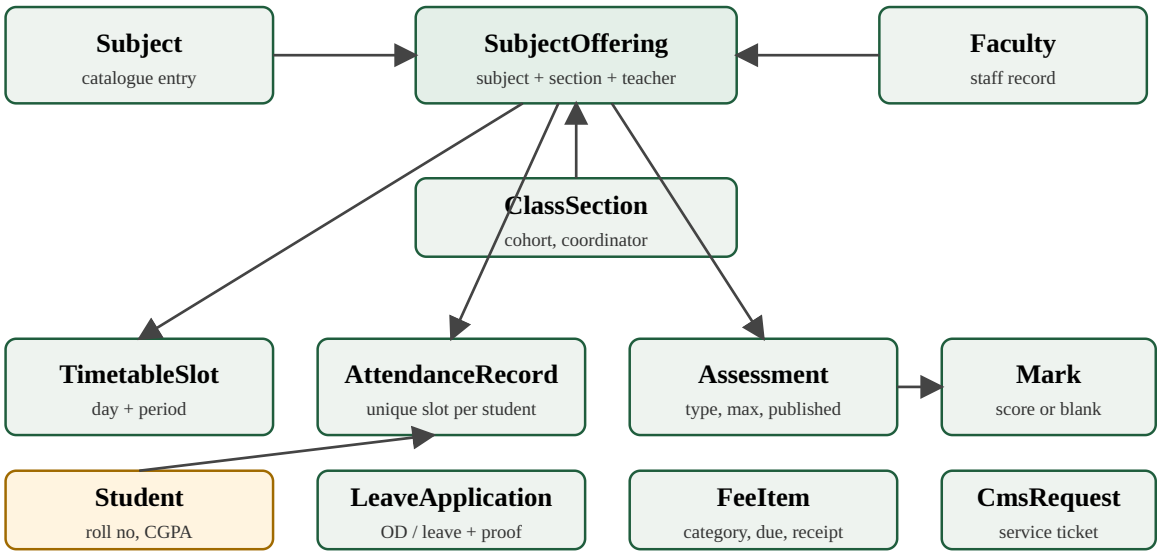


Figure 4.2: Core data model centred on the subject offering

4.3 MODULES

4.3.1 Authentication and account module

A single sign-in endpoint accepts a username and password and returns a signed token, the role, the display name and a flag that says whether the password must be changed. Accounts are created by the administrator, individually or from a spreadsheet, with a random ten-character temporary password. A student can later create a six-digit quick code from the settings page and use it, with the roll number, to sign in without the password. Five wrong codes lock the quick-code route for fifteen minutes.

4.3.2 Attendance module

The faculty portal offers two ways to mark. The one-period path suits the few minutes after a class: it pre-fills the roster as present and the teacher only changes the absentees. The whole-day path suits catching up on a backlog. Both end in the same upsert. Statuses are Present, Absent, On duty and Medical; on-duty and medical count as attended for the 75% eligibility calculation but are reported separately, so the register stays honest about what happened. The authority rule is shown in Figure 4.3.

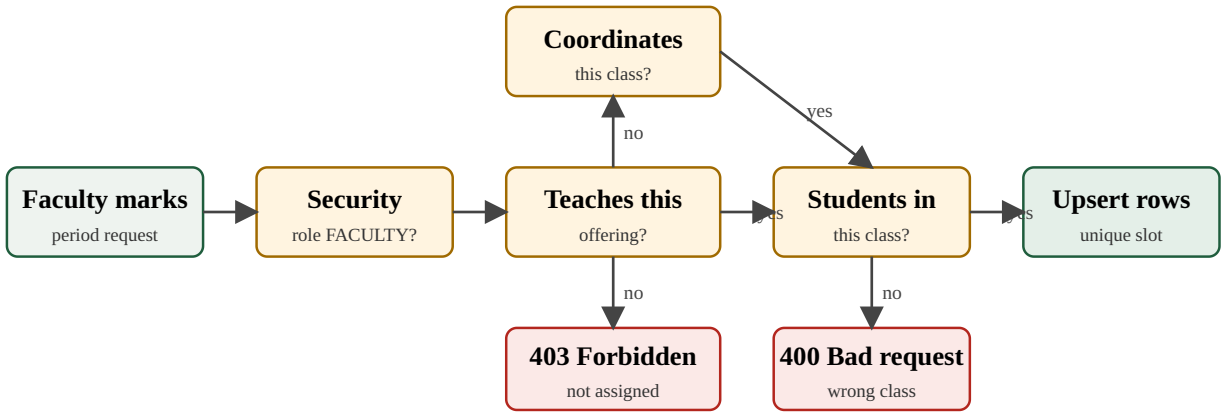


Figure 4.3: Authority check performed for every attendance request

4.3.3 Marks and examination module

Faculty enter scores for an assessment type such as Internal Assessment 1 or 2, Assignment, Unit Test, Group Presentation or, for laboratory subjects, Observation, Record, Model Exam and Viva. Each assessment has its own maximum mark. A blank field means unchanged, a lowering of the maximum below an already stored score is refused, and an assessment is visible to students only after it has been published. The student portal shows internal marks, the assessment breakdown, practical marks and the university results with the CGPA.

4.3.4 Fees and campus services module

A fee statement lists tuition, bus, hostel, examination and other items with due dates and flags overdue items. A student selects items and pays; the payment is simulated, a receipt number is issued and the statement is updated. A service queue accepts requests for fee receipts, hostel, transport, ID card, examination cell and library, each with a ticket number, and the administrator moves them through open, in progress and resolved.

4.3.5 Leave and on-duty module

A student submits a leave or on-duty application with the dates, a reason and an optional proof file. The staff who teach the student’s class see it in a review list. Approval of an on-duty application creates the corresponding on-duty entries in the attendance register, so the percentage is corrected without a second data entry.

4.3.6 Administration module

The administrator provisions accounts, creates departments, programmes and class sections, assigns coordinators, defines subject offerings with their weekly timetable slots, resolves service requests and reads the audit trail. A department can be opened to see its head, classes, staff and student count, and the head’s name can be edited from the same view.

4.3.7 Intelligence and Career Hub module

The scoring service calculates a weighted academic-risk score from attendance (0.35), CGPA (0.30), internal-mark trajectory (0.20) and the number of papers below 75% (0.15). Scores below 35 are low, 35 to 59 medium and 60 or more high. A placement-readiness score combines six dimensions and falls back to an academic proxy when a recorded score is missing. The Career Hub is a separate area of the student portal with its own black-and-gold theme; its tools work in the browser and keep the student’s progress on the device.

CHAPTER 5 – IMPLEMENTATION

5.1 DEVELOPMENT ENVIRONMENT

Table 5.1: Development environment

Item	Version / detail
Operating system	macOS (development) and Linux (test run for this report)
Java	17 (the packaged jar also runs on Java 21)
Node.js and npm	Node 24 with npm for the Vite front end
Python	3.13 for the scoring service
Build	Maven; the executable jar is about 72 MB
Ports	Front end 5173, API 8081, scoring service 8000
Browser used for the screenshots	Chromium 1194 driven by Playwright at 1440 x 900

5.2 STEP-BY-STEP IMPLEMENTATION

1. The domain model and repositories were written first, with the unique constraint on the attendance slot and the central subject-offering join.
2. The authentication service, token filter and security configuration were added, followed by the forced password change and the quick-code endpoints.
3. The attendance and marks services were written with the ownership rules and then covered by unit tests.
4. The student, faculty and administrator controllers exposed the services under three separate prefixes.
5. The React shell, theme, sign-in flow and the three portals were built against those endpoints.
6. The fee, leave, service, placement and examination modules were added, followed by the Career Hub.
7. The data seeder was written to create a complete institution with credentials written to a file that is excluded from version control.
8. The scoring service was built as a separate FastAPI application and connected through an HTTP client that degrades gracefully.
9. Finally the whole system was exercised through its REST interface and a real browser, and the results are recorded in Chapter 6.

5.3 FRONTEND COMPONENT STRUCTURE

The front end is organised as one page component per portal area and a small set of shared components. The application component moves the user through five stages: the splash screen with the college crest, the role picker, the sign-in form, the compulsory password change when

required, and the offer to create a quick code. After that the matching portal is mounted inside a common shell.

Table 5.2: Main front-end files

File	Responsibility
App.jsx	Stage machine from splash to portal; clears the token on sign-out
pages/Welcome.jsx	Splash, role picker, password and quick-code sign-in, quick-code set-up
components/Shell.jsx	Sidebar, page titles, theme switching, boxed Career Hub group
components/CodeSlots.jsx	Six-slot code input used for the quick code
components/LiquidEther.jsx	WebGL background used on sign-in and in the Career Hub
pages/StudentPortal.jsx, StudentExams.jsx, FeePortal.jsx	Student dashboard, examinations and fees
pages/CareerHub.jsx	Resume analyzer, GitHub analyzer, skill gap, courses, certifications, projects, study assistant
pages/FacultyPortal.jsx	Dashboard, attendance, marks entry, attention list, leave review
pages/AdminPortal.jsx	Access management, departments, courses and timetable, queues, audit
lib/api.js	The one place the browser talks to the server; token kept in memory only
lib/analyze.js, lib/pdf.js, lib/store.js	Resume scoring, PDF text extraction, per-student local store

The authentication token is kept in memory only. Nothing is written to local storage, so a shared laboratory machine does not retain a session after the tab is closed. On a 401 response that follows a request that carried a token, the client clears the token and returns to sign-in; on a failed sign-in, the same status simply means wrong credentials and the server’s message is shown.

5.4 SPRING BOOT SERVER AND API ROUTES

Three controllers expose the functionality. The student controller is mounted at /api/student, the faculty controller at /api/faculty and the administrator controller at /api/admin; the authentication controller handles /api/auth and /api/health. A global exception handler turns validation errors into 400 responses that name the field, missing records into 404, unsupported methods into 405 and unexpected failures into a neutral 500 message.

Table 5.3: Selected API routes

Route	Method	Purpose
/api/auth/login	POST	Password sign-in; returns token and mustChangePassword
/api/auth/login-code	POST	Quick-code sign-in for students

Route	Method	Purpose
/api/auth/change-password	POST	Change password (minimum 8 characters)
/api/faculty/attendance/period	POST	Mark one period
/api/faculty/attendance/day	POST	Mark a whole day
/api/faculty/marks	GET, POST	Read or save an assessment, draft or published
/api/faculty/leave-applications/{id}/approve	POST	Approve a leave or OD application
/api/student/attendance	GET	Register, per-subject and per-day roll-ups
/api/student/fees and /fees/pay	GET, POST	Statement and simulated payment
/api/student/placement/companies/{id}/apply	POST	Apply; eligibility re-checked on the server
/api/admin/users	GET, POST	List or provision accounts
/api/admin/offerings	GET, POST	Subject offerings with timetable slots
/api/admin/audit	GET	Most recent privileged actions

5.5 DATABASE MODEL AND CONSTRAINTS

The embedded H2 database in file mode was chosen because it is a single dependency with a file on disk, so nothing needs to be installed, yet it is a real relational database with real constraints. A spreadsheet cannot enforce the uniqueness of the attendance slot, and without that constraint a period can be marked twice and every percentage becomes wrong. The schema is created by Hibernate. Moving to PostgreSQL requires only a production profile that changes the connection URL; no service knows which database is behind the repositories.

Listing 5.1 – Attendance slot constraint

```
@Entity @Table(name = "attendance_records",
    uniqueConstraints = @UniqueConstraint(
        name = "uk_attendance_slot",
        columnNames = {"student_id", "subject_offering_id",
                       "class_date", "period"}))
public class AttendanceRecord {
    ...
    @Enumerated(EnumType.STRING) @Column(nullable = false, length = 16)
    private AttendanceStatus status;     // PRESENT, ABSENT, OD, MEDICAL
}
```

5.6 AUTHENTICATION AND AUTHORIZATION

Passwords are stored as BCrypt hashes. A provisioned account carries the mustChangePassword flag, so a temporary password shared by the office cannot remain in use. The temporary password is ten characters drawn from a secure random source over an alphabet that omits look-alike characters, with at least one letter and one digit. The quick code is hashed in the same way and is accepted only for student accounts.

Listing 5.2 – Route guards in the security configuration

```
.requestMatchers("/api/auth/login", "/api/auth/login-code",
                 "/api/health").permitAll()
.requestMatchers("/api/admin/**").hasRole("ADMIN")
.requestMatchers("/api/faculty/**").hasAnyRole("FACULTY", "ADMIN")
.requestMatchers("/api/student/**").hasAnyRole("STUDENT", "ADMIN")
.anyRequest().authenticated()
```

5.7 ATTENDANCE AUTHORITY AND ELIGIBILITY LOGIC

Every attendance write passes through a single ownership check. A staff member may mark an offering when he or she teaches it or when he or she is the coordinator of the class the offering belongs to; the administrator may always mark. Every student in the request must also belong to the class of the offering. The same check protects the day roster, which is readable only by the coordinator and the teachers of that class.

Listing 5.3 – Ownership check for attendance

```
boolean teaches = offering.getFaculty() != null
        && offering.getFaculty().getId().equals(faculty.getId());
ClassSection cs = offering.getClassSection();
boolean coordinates = cs != null && cs.getCoordinator() != null
        && cs.getCoordinator().getId().equals(faculty.getId());
if (!teaches && !coordinates) {
    throw new ResponseStatusException(HttpStatus.FORBIDDEN,
            "You are not assigned to this class");
}
```

Eligibility is the share of periods attended, where on-duty and medical entries count as attended. A subject appears in the student’s attention list when its percentage falls below 75%.

5.8 MARKS ENTRY AND PUBLISH GATE

The marks service applies four rules. First, a score outside the range zero to the maximum is rejected. Second, a blank score leaves the stored value unchanged, so a partial save cannot erase published marks. Third, the maximum cannot be lowered below a score that is already stored. Fourth, students see an assessment only when it is published; once it is published, later corrections are saved immediately and the response says so. The coordinator may enter marks for every subject of the class and a subject teacher only for his or her own subject.

5.9 ACADEMIC RISK SCORE

The risk score is a weighted sum of four factors and each response carries the contribution of every factor. The curve for attendance is steep just below 75% and flat above 85%. The headline names the largest driver, for example that attendance is below the requirement, because an actionable statement is more useful than a bare number. The weights are held in one dictionary so that the college can tune them when real outcome data is available. The score is explicitly not a trained model.

Table 5.4: Weights used by the academic-risk score

Factor	Weight	What it reads
Attendance	0.35	Overall percentage against the 75% line
Academic	0.30	CGPA against placement thresholds
Trajectory	0.20	IA1 to IA2 movement averaged across papers
Eligibility	0.15	Number of papers below 75%

5.10 FEE PAYMENT AND RECEIPT LOGIC

A payment request carries the identifiers of the selected fee items and a mode. The service looks the items up on the requesting student’s own statement, rejects an identifier that does not belong to that statement, rejects an item that is already paid, marks each item paid, issues a receipt number and returns the updated statement. Payment is simulated; no card or bank detail is collected anywhere in the application.

5.11 LEAVE AND ON-DUTY WORKFLOW

The application is validated for date order and for the proof file type and size, saved in PENDING state and shown to the staff who teach the class. Approval changes the state to APPROVED, records the reviewer and, for on-duty applications, creates on-duty attendance entries for the dates. A second approval of the same application is refused with a conflict response.

5.12 ERROR HANDLING AND VALIDATION

Request objects use Bean Validation, so a missing or malformed field produces a 400 response with the field name. Malformed JSON, an unknown route and an unsupported method produce 400, 404 and 405 with a plain explanation. The front end displays the server’s message beside the form instead of a generic failure.

5.13 DEPLOYMENT CONSIDERATIONS

For a trial the three services run on one machine. The data seeder has two modes: a minimal mode that creates only the administrator account and a full mode that creates the whole institution and writes the generated sign-in details to a file. For production the file database should be replaced by PostgreSQL, the secret used to sign tokens must be supplied through the environment, the H2 console route must be disabled, the application must be served over HTTPS and regular backups must be scheduled.

CHAPTER 6 – SYSTEM TESTING

6.1 TESTING OBJECTIVE

The objective of testing is to show that the rules claimed for the system actually hold when it runs: that only the right people can sign in and change records, that each rule in the attendance, marks, fee and leave workflows is enforced by the server and not only by the screen, and that bad input is refused with a clear message. The tests in this chapter were executed on 22 October 2026 against the packaged application running with the fully seeded institution, using its REST interface for the rule checks and a real browser for the screens in Chapter 7. The “Actual result” columns are the responses that were recorded during that run.

6.2 TEST CASE FORMAT

Each table lists the test case number, the input or action, the expected result, the actual result and the status. HTTP status codes are quoted as returned by the server. Accounts used were the administrator, the class coordinator of CSE II-E, a subject teacher of the same class and the student account of the first author.

Table 6.1: Test documentation – Valid and invalid login

TC	Input / action	Expected result	Actual result	Status
TC1	Administrator signs in with the correct password	200, token issued, role ADMIN	200, token issued, role ADMIN	Pass
TC2	Administrator signs in with a wrong password	401 with a neutral message	401 – “That username and password combination isn’t recognised”	Pass
TC3	Unknown username	401, same neutral message	401, same message (the response does not reveal which part was wrong)	Pass
TC4	Blank username and password	400 naming both fields	400 – “username: must not be blank; password: must not be blank”	Pass

Table 6.2: Test documentation – Account provisioning and first login

TC	Input / action	Expected result	Actual result	Status
TC5	Administrator creates a student account	200, random temporary password issued	200, user created, 10-character temporary password returned once	Pass
TC6	Create a second account with the same username	409 conflict	409 – “That username is already in use”	Pass
TC7	Create an account without a name	400 naming the field	400 – “name: must not be blank”	Pass
TC8	New student signs in with the temporary password	200 with mustChangePassword true	200, mustChangePassword = true	Pass

Table 6.3: Test documentation – Password change and quick code

TC	Input / action	Expected result	Actual result	Status
TC9	Change password to a 5-character value	400	400 – “Use at least 8 characters”	Pass
TC10	Change password with a wrong current password	400	400 – “Current password is incorrect”	Pass
TC11	Change password correctly, then sign in again	200, flag cleared	200 – “Password updated”; next sign-in shows mustChangePassword = false	Pass
TC12	Create a quick code with 5 digits	400	400 – “The code must be exactly 6 digits”	Pass
TC13	Create a 6-digit quick code, then sign in with the code	200 both times	200 – “Quick code saved”; code sign-in returned a token	Pass
TC14	A faculty account tries to create a quick code	Refused	400 – “The quick code is available for student accounts only”	Pass

Table 6.4: Test documentation – Quick-code lockout

TC	Input / action	Expected result	Actual result	Status
TC15	Six consecutive wrong codes for one student	Locked after the fifth	Responses 401, 401, 401, 401, 401, then 429	Pass
TC16	Correct code while the lock is active	429, advice to use the password	429 – “Too many wrong codes. Sign in with your password instead, or try again in a few minutes.”	Pass

Table 6.5: Test documentation – Who may mark attendance

TC	Input / action	Expected result	Actual result	Status
TC17	Class coordinator marks a period of a subject taught by another teacher	200 (coordinator controls the whole class)	200 – “Attendance saved for period 3”	Pass
TC18	Subject teacher marks a period of his or her own subject in the same class	200	200 – “Attendance saved for period 2”	Pass
TC19	Same subject teacher marks a period of another teacher’s subject	403	403 – “You are not assigned to this class”	Pass
TC20	Subject teacher’s class list shows editable flags	Only own subject editable	Eight subjects returned editable = false and the teacher’s own subject editable = true	Pass

Table 6.6: Test documentation – Attendance validation and duplicate marking

TC	Input / action	Expected result	Actual result	Status
TC21	Mark the same period again for the same students	Update, no duplicate	200; the unique slot constraint turned it into an update and the student’s attendance stayed at 91%	Pass

TC	Input / action	Expected result	Actual result	Status
TC22	Period number 9	400	400 – “period: must be less than or equal to 8”	Pass
TC23	Include a student from another class	400 naming the student	400 – “CSE201 is not in BECSE Y2-E”	Pass
TC24	Use an unknown status value	400	400 – request body could not be read	Pass

Table 6.7: Test documentation – Marks entry rules

TC	Input / action	Expected result	Actual result	Status
TC25	Score 25 for an assessment out of 20	400	400 – “Score 25.0 is outside 0–20”	Pass
TC26	Lower the maximum to 10 when 18 is already stored	400	400 – “A score of 18.0 is already recorded, above the new maximum of 10.”	Pass
TC27	Save a blank score for a student	Stored value unchanged	200; the sheet still showed 18.0 for that student	Pass
TC28	Subject teacher enters marks for another teacher’s subject	403	403 – “You are not assigned to this class”	Pass
TC29	Coordinator enters marks for another teacher’s subject	200	200 – saved	Pass

Table 6.8: Test documentation – Publish marks

TC	Input / action	Expected result	Actual result	Status
TC30	Teacher publishes a unit test for the class	200, visible to students	200 – “Marks published and visible to students”	Pass
TC31	Student opens the assessments list afterwards	200 with the components	200; components and maximum marks listed for every subject	Pass
TC32	Edit an already published assessment	Saved, student told	200 – “Saved. This assessment is already published, so students see the change.”	Pass

Table 6.9: Test documentation – Fee statement and payment

TC	Input / action	Expected result	Actual result	Status
TC33	Student opens the fee statement	200 with totals	200 – billed 2,28,200; paid 2,04,000; due 24,200; overdue 16,000; 11 items	Pass
TC34	Student pays one item (UPI)	200, receipt issued, totals change	200; paid rose by 750 and due fell by 750	Pass
TC35	Pay the same item again	409	409 – “CSE association and symposium fee is already paid.”	Pass
TC36	Pay with an empty list of items	400	400 – “feeIds: must not be empty”	Pass
TC37	Pay an item that belongs to another student	404	404 – “That fee isn’t on your statement.”	Pass

Table 6.10: Test documentation – Leave and on-duty workflow

TC	Input / action	Expected result	Actual result	Status
TC38	Student submits an on-duty application	200, status PENDING	200, status PENDING	Pass
TC39	From-date later than to-date	400	400 – “From date must be on or before to date”	Pass
TC40	Coordinator opens the pending list	Contains the new application	200; application listed with two older pending ones	Pass
TC41	Staff of the class approve it	200, APPROVED	200 – approved	Pass
TC42	Approve the same application again	409	409 – “This application is no longer pending”	Pass

Table 6.11: Test documentation – Campus service requests

TC	Input / action	Expected result	Actual result	Status
TC43	Student raises a library request	200, ticket number, OPEN	200 – ticket EEC-15043, status OPEN	Pass
TC44	Administrator resolves it with a note	200, RESOLVED	200 – RESOLVED	Pass
TC45	Administrator sends an unknown status value	400 with a clear message	500 – generic server message	Fail (defect D1)

Table 6.12: Test documentation – Placement eligibility

TC	Input / action	Expected result	Actual result	Status
TC46	Apply to a drive open to ECE only	403 with the reason	403 – “You don’t meet the criteria: Open to ECE only”	Pass
TC47	Apply to an eligible drive	200	200 – “Application submitted”	Pass
TC48	Apply to the same drive again	409	409 – “You have already applied to this company”	Pass

Table 6.13: Test documentation – Unauthorized access

TC	Input / action	Expected result	Actual result	Status
TC49	Student calls an administrator route	403	403	Pass
TC50	Student calls a faculty route	403	403	Pass
TC51	Faculty calls a student route	403	403	Pass
TC52	Request without a token	Refused	403	Pass
TC53	Request with a forged token	Refused	403	Pass
TC54	Teacher of another department opens a class roster	403	403 – “You are not assigned to this class”	Pass

Table 6.14: Test documentation – Malformed requests

TC	Input / action	Expected result	Actual result	Status
TC55	Invalid JSON body at sign-in	400	400 – “The request body couldn’t be read. Check the fields and try again.”	Pass
TC56	Unknown route	404	404 – “That page or endpoint doesn’t exist.”	Pass
TC57	Wrong HTTP method	405	405 – “That action isn’t supported here.”	Pass

Table 6.15: Test documentation – Account access and password reset

TC	Input / action	Expected result	Actual result	Status
TC58	Administrator revokes a student’s access	200	200	Pass
TC59	Revoked student signs in	Refused with a clear message	403 – “This account is deactivated. Contact the administration office.”	Pass
TC60	Administrator grants access again; student signs in	200	200 for both	Pass
TC61	Administrator resets the password	Temporary password, change forced	200 – “Share this once. The user must change it at next sign-in.” (the temporary value is the role default; see D2)	Pass
TC62	Administrator deletes the test account	200	200	Pass

Table 6.16: Test documentation – Administrative actions and audit trail

TC	Input / action	Expected result	Actual result	Status
TC63	Reassign the class coordinator of CSE II-E	200	200, class summary returned	Pass
TC64	Edit a department’s name	200	200, department returned	Pass
TC65	Read the audit trail	Actions listed newest first	200 – 23 entries including DEPARTMENT_UPDATED, CLASS_COORDINATOR_UPDATED, USER_DELETED, PASSWORD_RESET, ACCESS_GRANTED, ACCESS_REVOKED	Pass

Table 6.17: Test documentation – Scoring service availability

TC	Input / action	Expected result	Actual result	Status
TC66	Scoring service stopped; open the administrator dashboard and Risk page	Pages render without an error	Dashboard and Risk page rendered; risk counts showed zero; health reported scoringService DOWN	Pass

TC	Input / action	Expected result	Actual result	Status
TC67	Scoring service started; open the Risk page and the learning path	Real results	Risk page listed one high-risk student in ECE and one in IT; learning path listed three recommendations with reasons	Pass

Table 6.18: Test documentation – Database restore drill

TC	Input / action	Expected result	Actual result	Status
TC68	Copy the database file, stop and restart the server on the copy	All records identical	Users 85 = 85; attendance of the student 499 periods, 454 attended, 91% (unchanged); fees billed, paid and due unchanged; the seeder did not run again	Pass

6.3 TESTING TYPES

Functional and negative testing of the REST interface produced 68 recorded test cases; 67 passed and 1 failed. The failure (D1) is an invalid service-status value that currently returns a generic 500 response instead of a 400 message; the record is not changed. A second observation (D2) is that the administrator’s password reset still issues a role default value, which the user is forced to change but which is less random than the temporary passwords issued for new accounts. Both items are listed again as limitations in Chapter 8.

Unit testing covers the rules that are most likely to be broken by a later change. Eleven JUnit tests pass: three in the faculty-service test class, two in the attendance-service test class, four in the marks-service test class and two for the temporary-password generator. They check, among other things, that a coordinator and a subject teacher are treated differently, that a student outside the class is rejected, that a blank score does not erase a published mark and that generated passwords always contain letters and digits and never the look-alike characters.

Usability and visual testing were done in a real Chromium browser at a width of 1440 pixels. Every screen shown in Chapter 7 was opened by signing in through the interface. Layout at phone width, the expiry of the fifteen-minute lock and animation performance on low-end machines were not tested and are noted as limitations.

CHAPTER 7 – OUTPUT AND EXPLANATION

This chapter presents the implemented portals through screenshots taken from the running system. The student screens belong to the account of the first author (register number 310625104288, CSE II-E); the faculty screens belong to the class coordinator of that class and to a subject teacher; the administrator screens show the whole institution. The records shown belong to the trial institution created by the data seeder: five departments, ten classes, about sixty students and twenty-five staff members. Each screen is described briefly.

7.1 WELCOME AND ROLE SELECTION

The first screen after the splash shows the college crest over the animated background and asks how the person is signing in: Student, Faculty or Administrator. The choice decides which sign-in form and which portal follow. Keeping the role choice before the credentials means that the same username format can be used for staff and students without confusion.

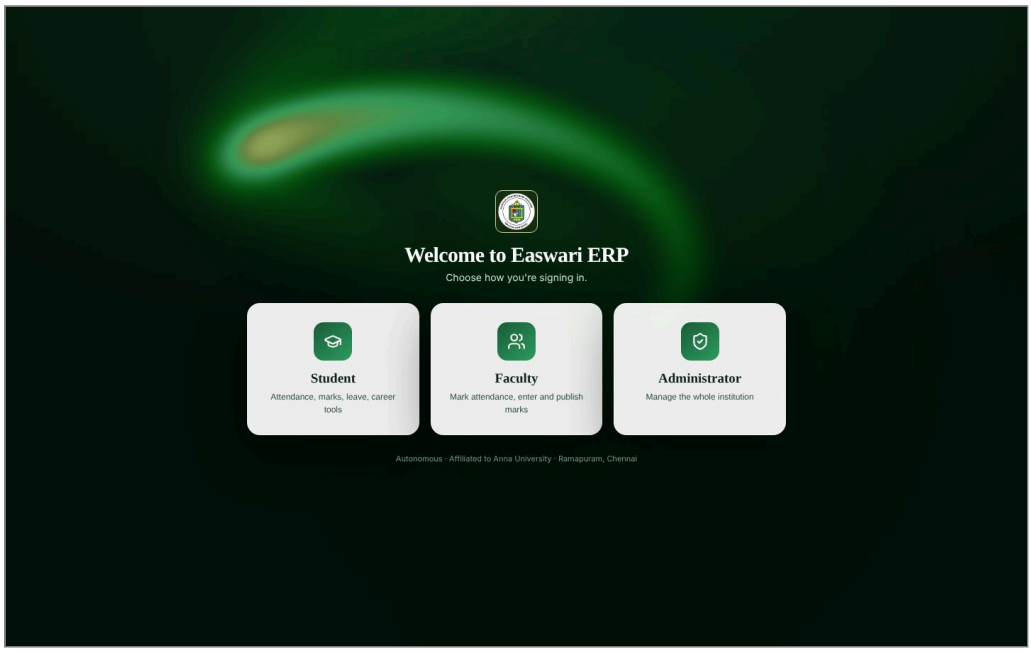


Figure 7.1: Welcome and role selection

7.2 STUDENT SIGN-IN

The student sign-in asks for the register number and the password and offers a second tab for the quick code. A first-time user is reminded that the temporary password given by the administrator will have to be replaced at the first login. Failed attempts show the neutral message recorded in test case TC2, which does not reveal whether the register number or the password was wrong.

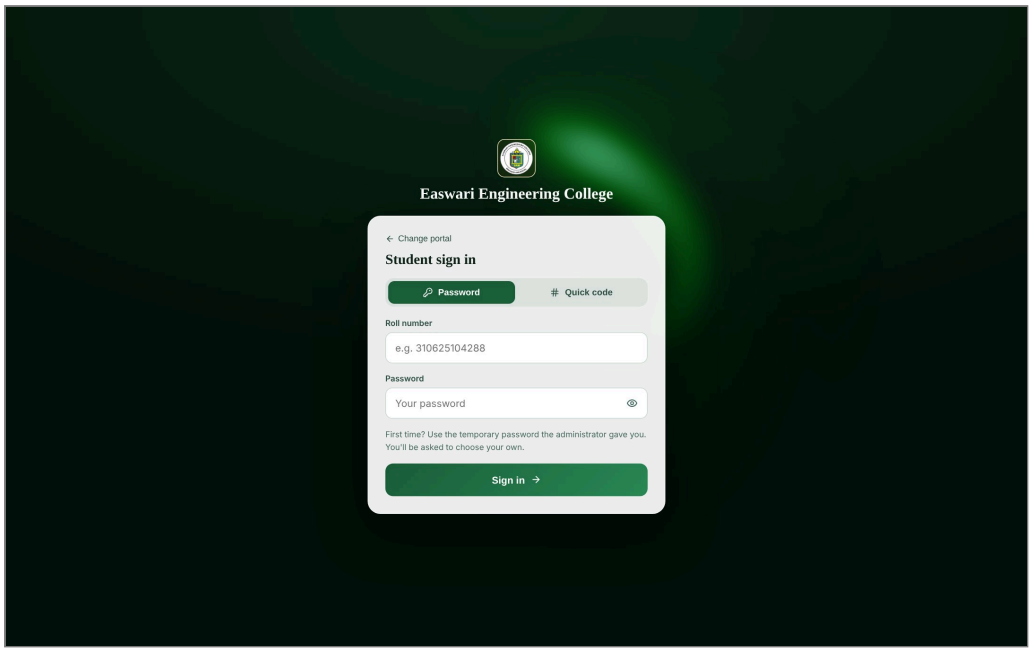


Figure 7.2: Student sign-in

7.3 QUICK-CODE SIGN-IN

After the first successful sign-in a student can create a six-digit quick code. The quick-code tab shows six separate slots for the digits and the register number field. The code works only for student accounts, is stored as a BCrypt hash and is locked after five wrong attempts, as shown in test cases TC15 and TC16.

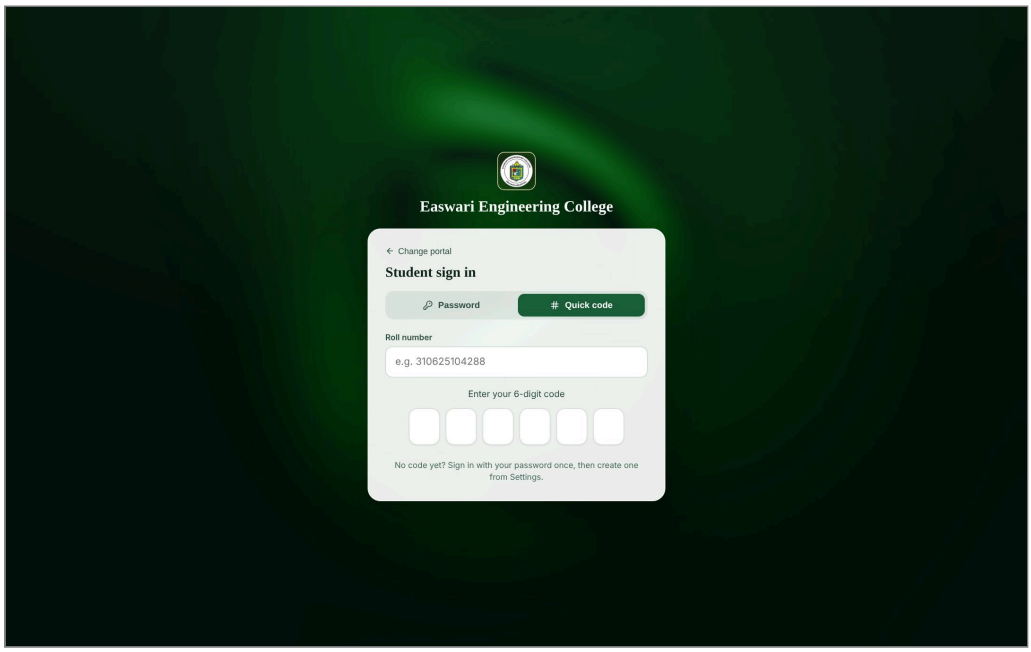


Figure 7.3: Quick-code sign-in

7.4 ADMINISTRATOR DASHBOARD

The administrator’s dashboard summarises the institution: students, teaching staff, institution-wide attendance and open service requests, followed by a chart of attendance for each department and a short list of departments with high-risk students. The figures are read from the database at the time the page opens, so there is no separate report to prepare.

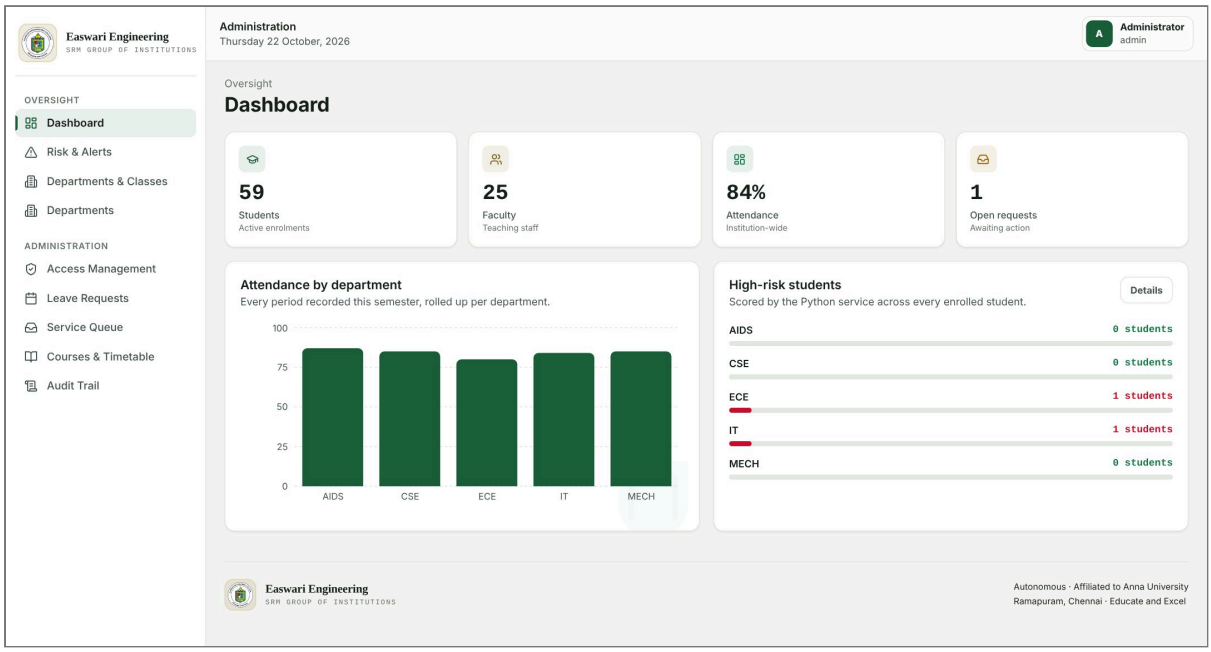


Figure 7.4: Administrator dashboard

7.5 ACCESS MANAGEMENT

Access Management lists every account with its role, department and creation time and offers the actions Reset, Revoke and Delete. Filters separate students, faculty and administrators and a search box finds a person by name or username. The toolbar provides a spreadsheet template, bulk import and the New account button. Every action on this page is written to the audit trail.

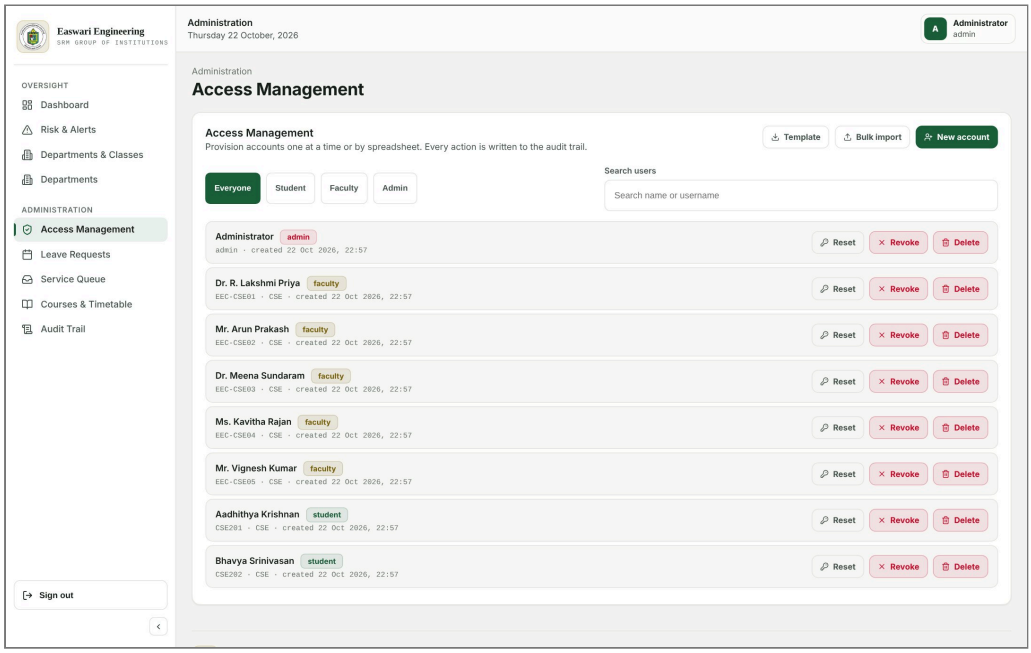


Figure 7.5: Access management

7.6 CREATE AN ACCOUNT

The account form asks for the role, the username, the full name and e-mail and, for a student, the programme and the class. Only classes that exist are listed so that a student always lands in a real roster. The note at the bottom explains that a temporary password is issued automatically and that the account holder must change it at the first sign-in. The generated password is shown once, with a copy button.

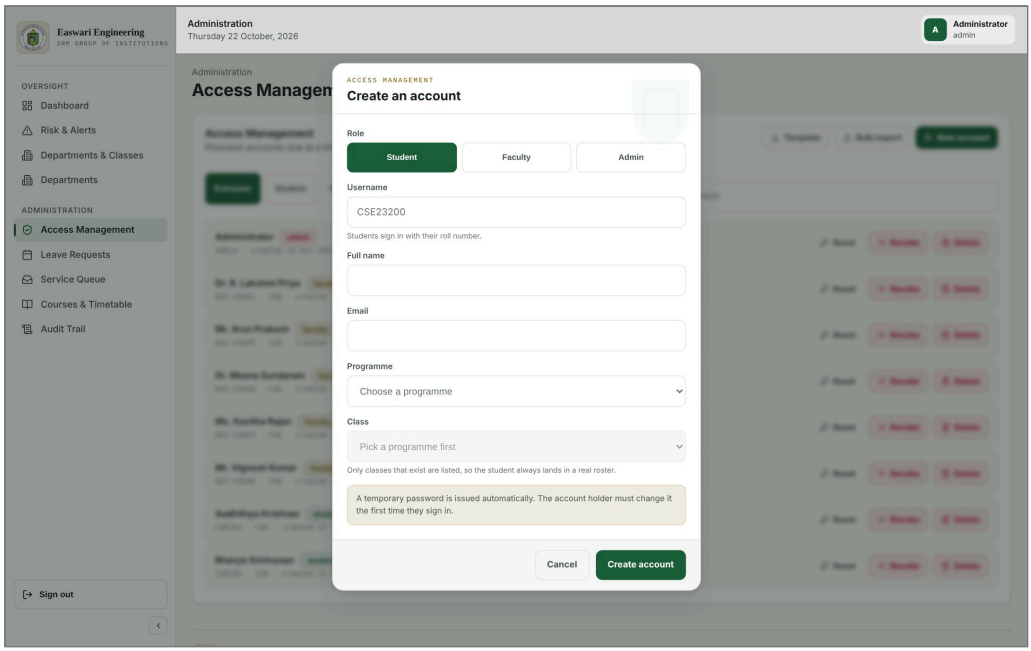


Figure 7.6: Create an account

7.7 RISK AND ALERTS

The Risk and Alerts page counts high-risk students in each department, scored on attendance, results, trajectory and eligibility. The scale at the bottom of the page states that the score is weighted and explainable and not a prediction, and that faculty see the individual reasons in their own portal. In this run the scoring service was running and flagged one student each in ECE and IT.

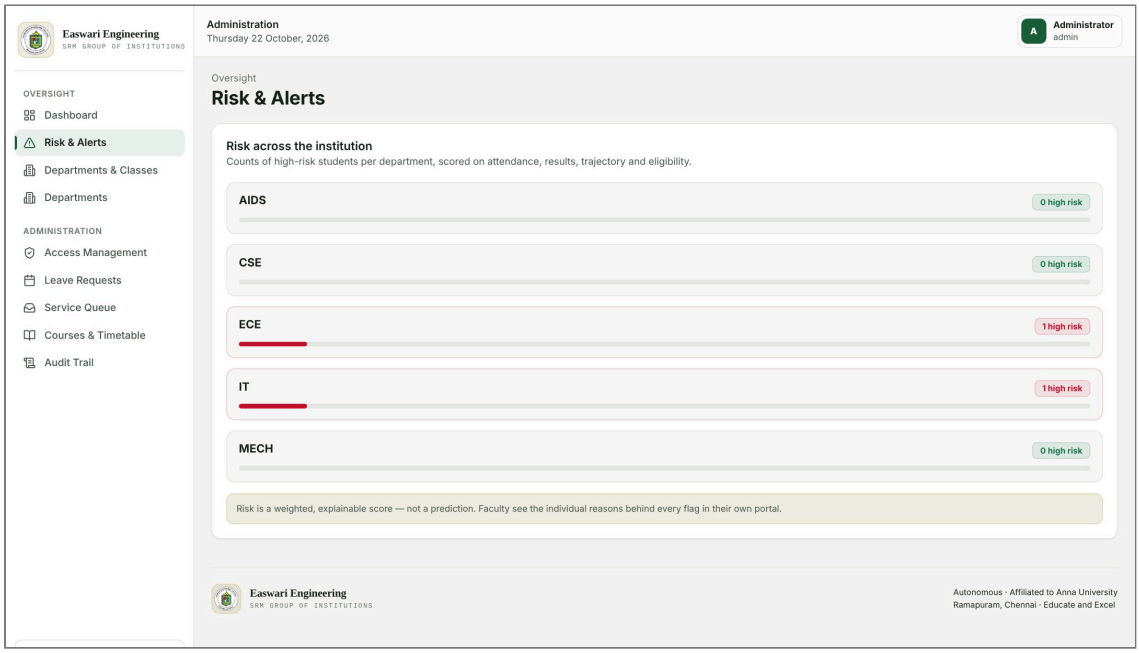


Figure 7.7: Risk and alerts

7.8 DEPARTMENTS AND CLASSES

This page lists the five departments with their codes, head of department and an Open link, and below them every class section with its year, section, number of students and the faculty member who is its coordinator. The coordinator of a class can be changed from the drop-down, which is how the rule in Section 4.3.2 is assigned.

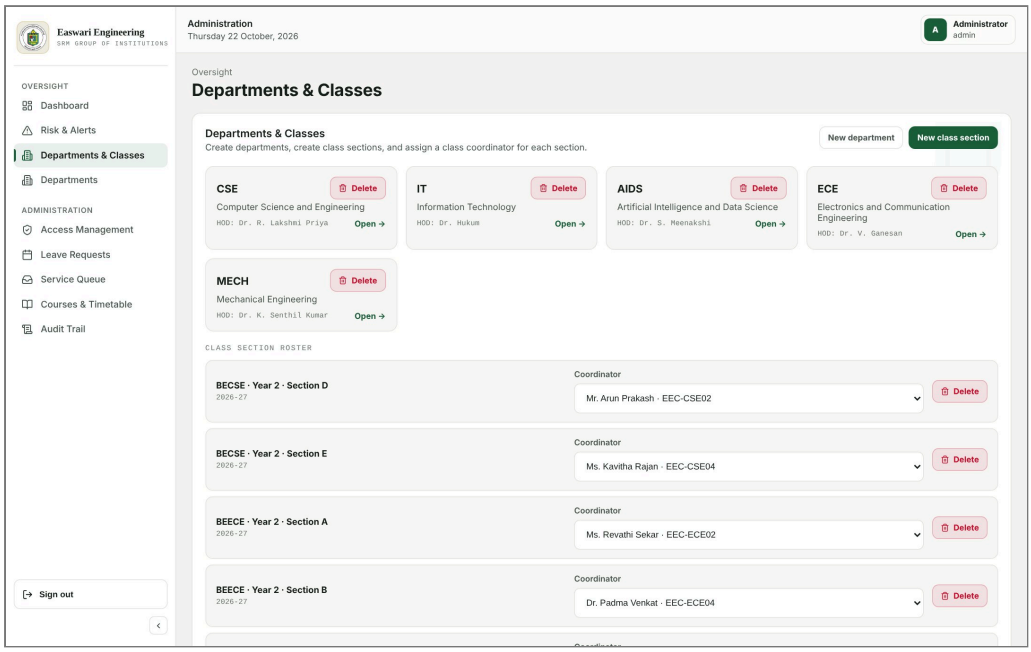


Figure 7.8: Departments and classes

7.9 DEPARTMENT DETAIL AND HOD EDIT

Opening a department shows its head, the number of programmes, classes, faculty and students, the classes with their coordinators and the faculty list with each member's designation, staff code and the subjects and classes he or she handles. The Edit name / HOD button changes the department's name or head; the change is recorded in the audit trail (test case TC64).

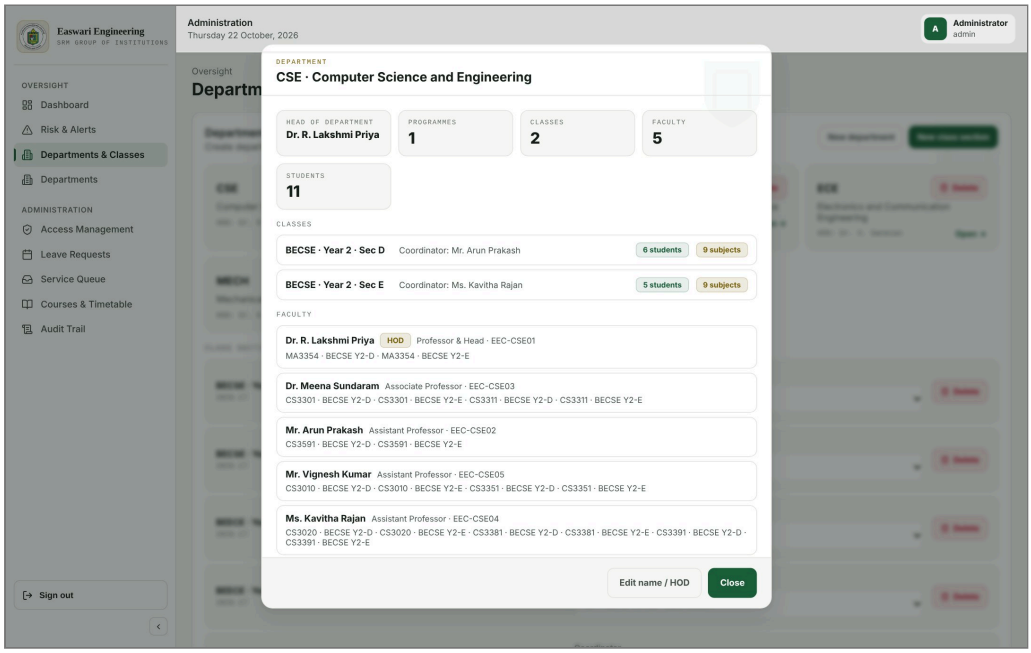


Figure 7.9: Department detail and HOD edit

7.10 COURSES AND TIMETABLE

Courses and Timetable has two parts. The upper part holds the programmes of the institution. The lower part lists the courses offered to each class with the credits, the weekly periods and the teacher chosen from a drop-down. Assigning a teacher or defining a new offering creates the subject offering to which attendance and marks will later be attached; overlapping periods are refused with a conflict message.

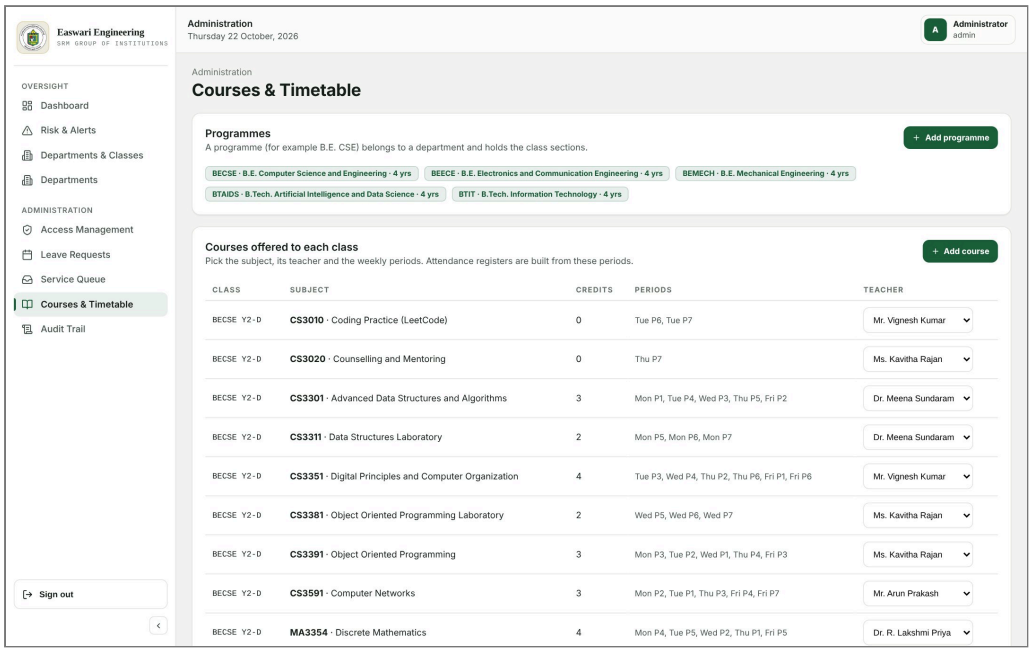


Figure 7.10: Courses and timetable

7.11 LEAVE REQUESTS (ADMINISTRATION)

The administrator sees the leave and on-duty applications of the whole institution with the student, class, type, dates, reason and status. Pending, approved and rejected applications are distinguished by colour. This view is for oversight; the day-to-day review is done by the staff of the class, as shown in Section 7.18.

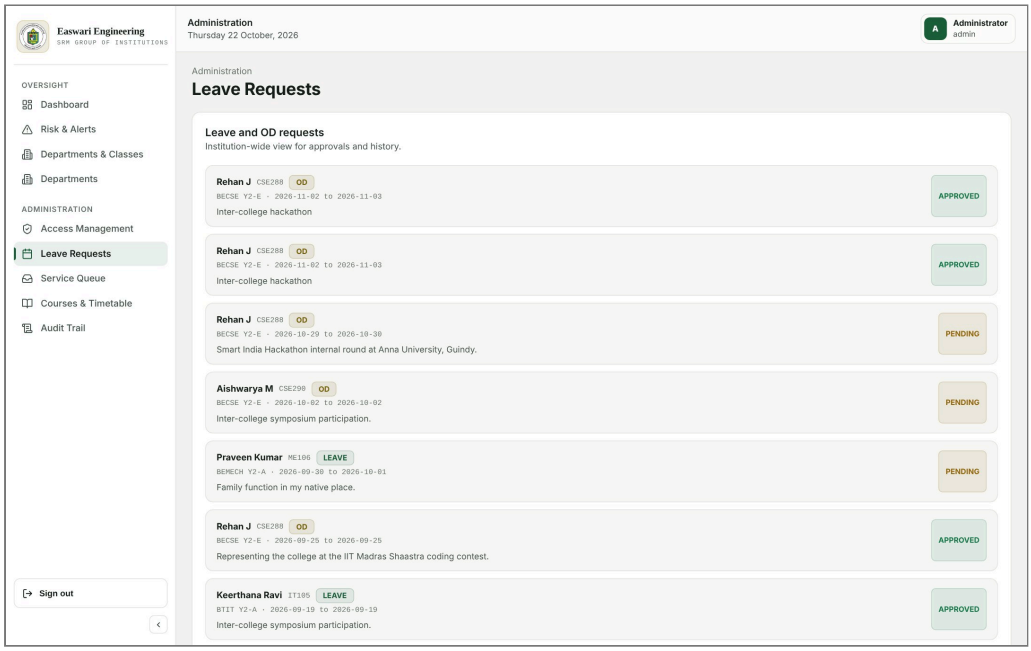


Figure 7.11: Leave requests (administration)

7.12 SERVICE QUEUE

The service queue lists open campus requests with their ticket number, the student and the date. The administrator can decline a request or mark it resolved with a note. When the queue is empty, the page says so instead of showing a blank area. The status values accepted by the server are open, in progress, resolved and rejected.



Figure 7.12: Service queue

7.13 AUDIT TRAIL

The audit trail lists the most recent privileged actions with the action name, the actor, the time and the target. Typical entries are password changes, access revoked, access granted, password reset, coordinator updated and department updated. It answers the question of who did what and when, which a spreadsheet cannot.

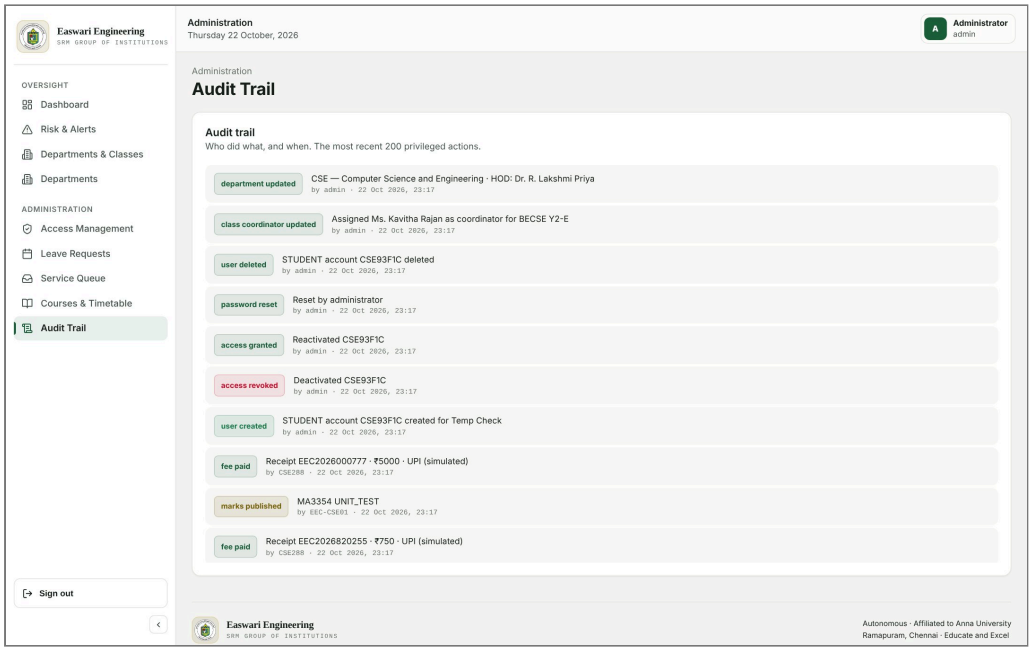


Figure 7.13: Audit trail

7.14 FACULTY DASHBOARD

The faculty dashboard opens on the class that the member coordinates. It shows the number of classes and students, the average attendance and the number of students below 75%, a chart of attendance for each class that the member teaches, a priority list of students who need attention and a list of the member's classes with the subject, the attendance and the at-risk count. A Mark attendance shortcut leads to the register.

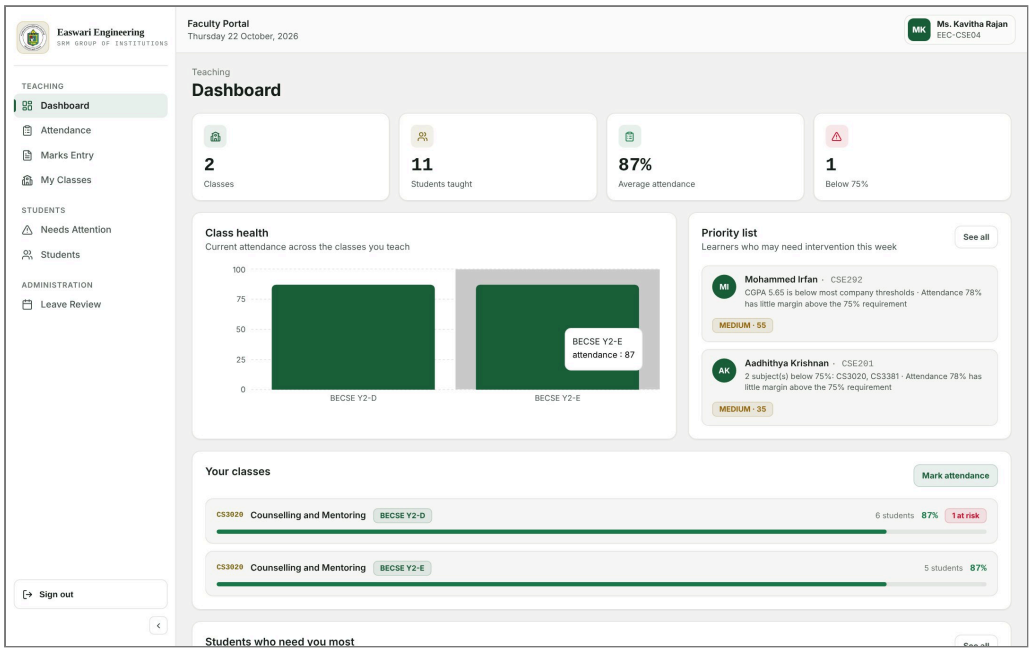


Figure 7.14: Faculty dashboard

7.15 ATTENDANCE ENTRY BY THE CLASS COORDINATOR

The coordinator of the class sees a banner that says the member may mark and change every period of the class. The page offers the class, the date and the choice between marking one period and the whole day. The seven periods of the day are shown as tabs with their subject codes. The roster lists the students with Present, Absent, On duty and Medical buttons, pre-filled as present, and the Save this period button writes the rows.

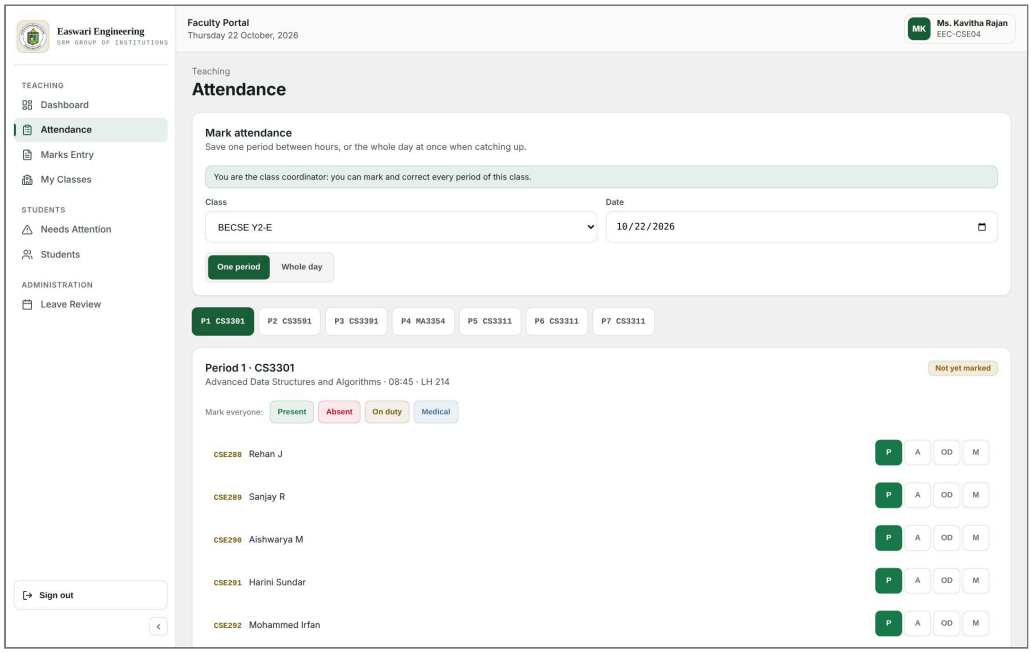


Figure 7.15: Attendance entry by the class coordinator

7.16 ATTENDANCE ENTRY BY A SUBJECT TEACHER

A subject teacher who is not the coordinator sees a notice that only his or her own subject's periods can be marked and that the other periods are view-only. The periods of other teachers are shown greyed in the tab row. The server enforces the same rule: an attempt to save another teacher's period returns 403 (test case TC19).

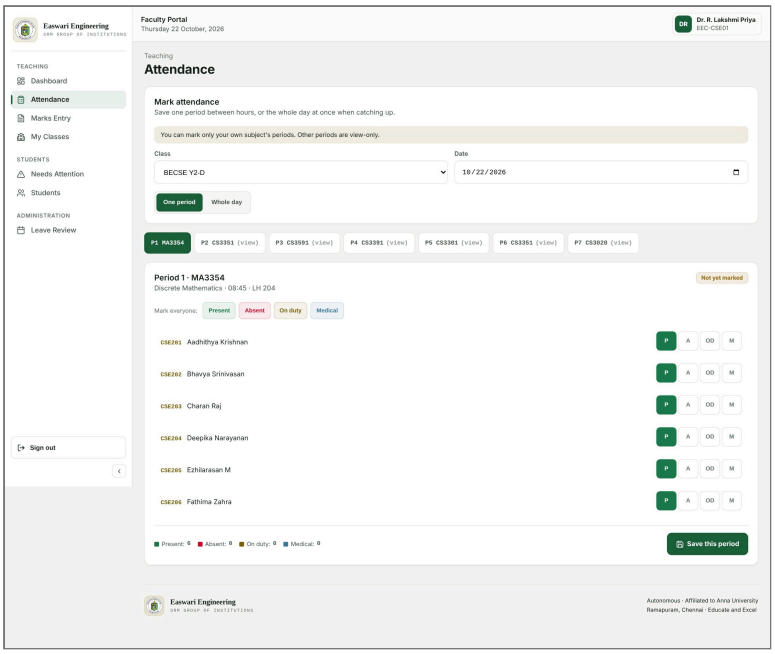


Figure 7.16: Attendance entry by a subject teacher

7.17 MARKS ENTRY

Marks Entry lets the faculty pick the class, the subject, the assessment and the maximum marks and then type a score for each student. A draft is private to the teacher; the Publish button makes it visible to students. For laboratory subjects the assessment list includes observation, record, model examination and viva. Scores are checked against the maximum and a blank field leaves the stored mark unchanged.

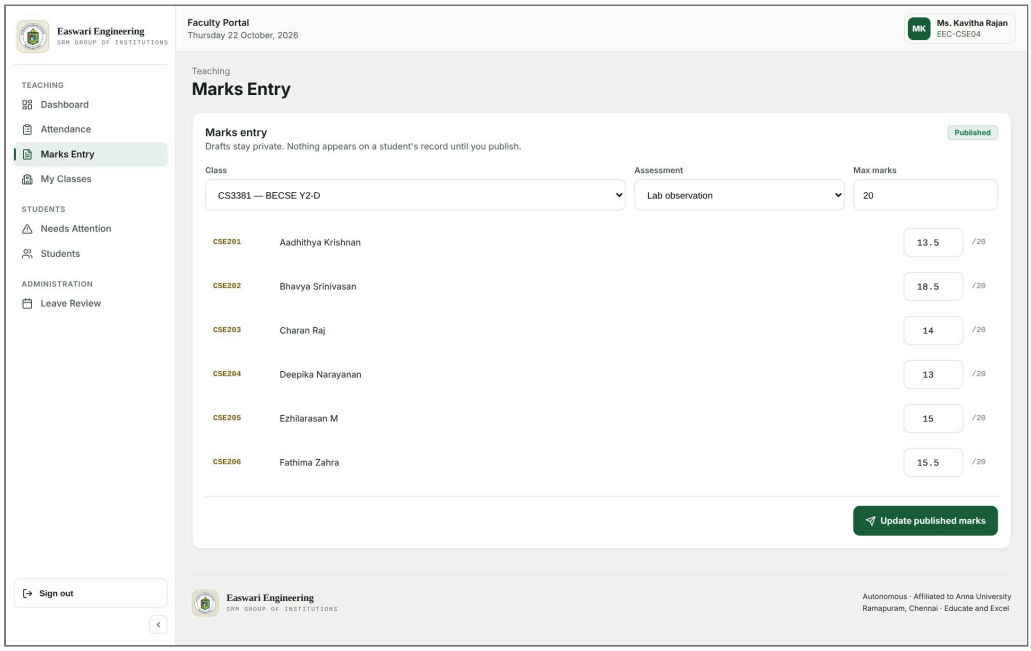


Figure 7.17: Marks entry

7.18 LEAVE REVIEW

Leave Review lists the pending leave and on-duty applications of the classes that the member teaches. Each card shows the student, the dates, the reason and any proof, with a field for a rejection note and the Reject and Approve buttons. Approving an on-duty application also corrects the attendance register for those dates.

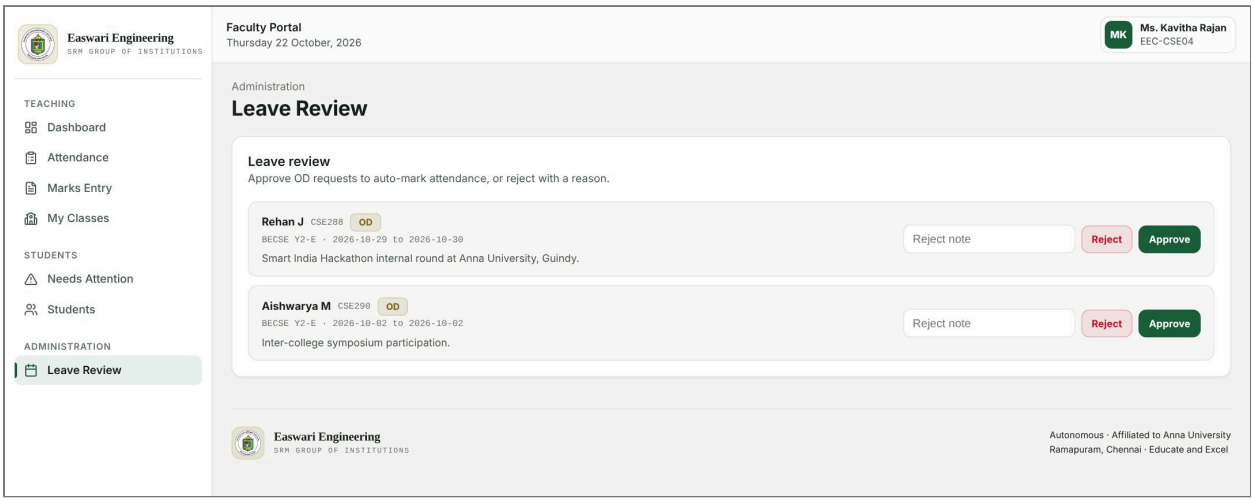


Figure 7.18: Leave review

7.19 STUDENT DASHBOARD

The student dashboard begins with the student's profile: name, register number, programme, year, semester, section, batch, class coordinator and mentor. Four figures follow: CGPA, attendance, fees due and open requests, each linked to its page. The attendance-by-subject table shows periods attended, periods held and the percentage with a bar and a mark at 75%. The Needs your attention list gathers overdue fees, pending requests and the internship closing soonest, and the internal marks of the semester are summarised below.

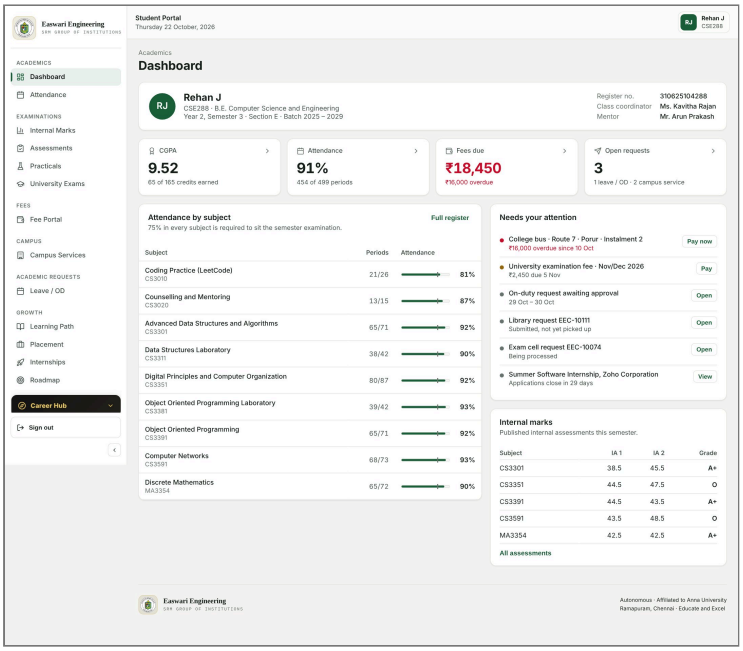


Figure 7.19: Student dashboard

7.20 ATTENDANCE REGISTER

The attendance page repeats the percentage, the periods attended and the periods missed, and shows the register as a grid with one small cell for each period, coloured for present, absent, on duty and medical. The student can filter by subject and switch between a per-period view and a per-day view. The chart below explains how the periods were recorded. The data is the same set of rows that the faculty marked.

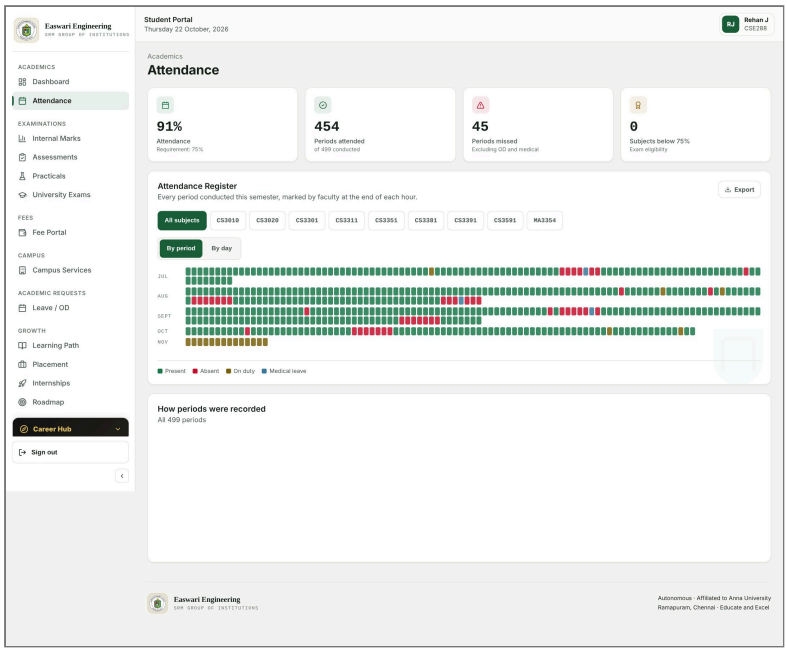


Figure 7.20: Attendance register

7.21 INTERNAL MARKS

The Internal Marks page lists each subject with its credits, the marks of the first and second internal assessments, the average percentage and the grade, and states the CGPA and the credits earned. Only published marks appear. A note at the bottom says that final grades follow the university examination.

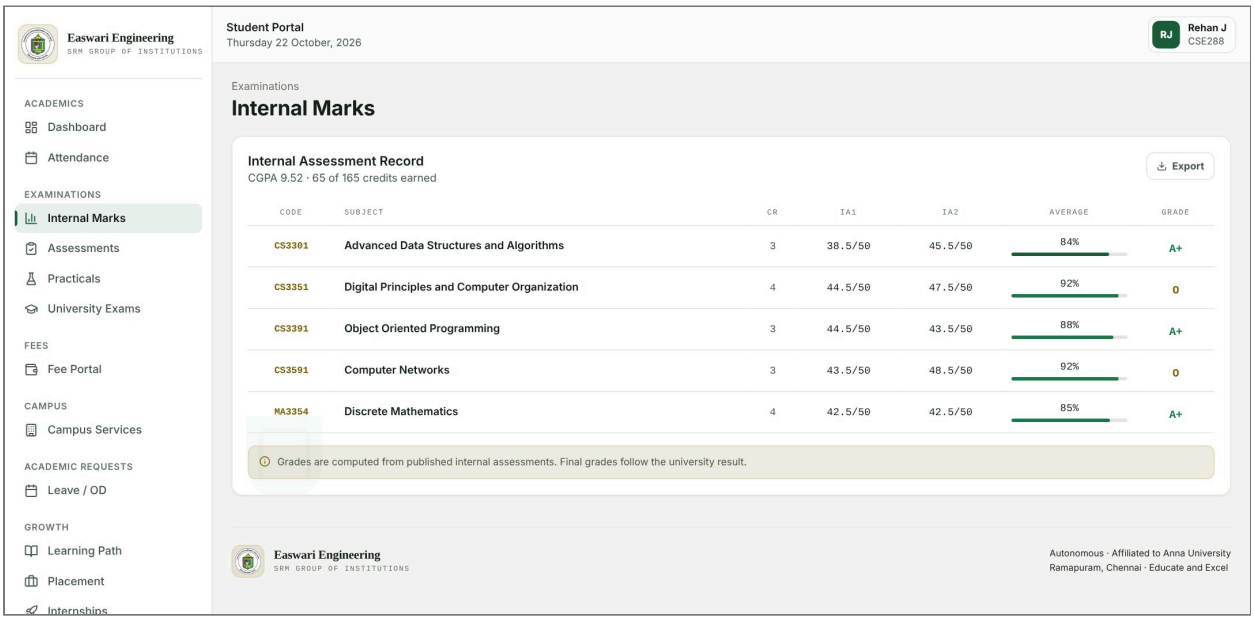


Figure 7.21: Internal marks

7.22 ASSESSMENTS

The Assessments page breaks each subject into its components: assignment, internal assessments, unit test and, where relevant, a group presentation, each with the score and the maximum. A note explains that only published marks are shown and that a test whose marks have not yet been released does not appear until the teacher publishes it.

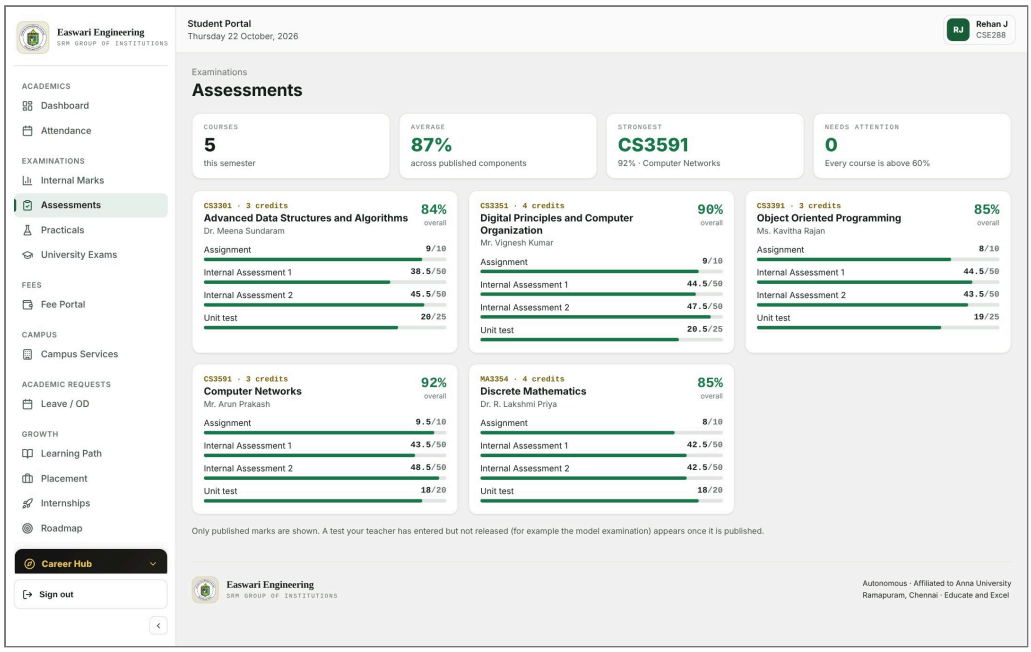


Figure 7.22: Assessments

7.23 PRACTICALS

The Practicals page shows the semester’s laboratories with their observation, record, model-examination and viva marks, and the completed practical examinations of earlier semesters with the internal, external and total marks and the grade. It separates laboratory evaluation from theory so that a student can see where the marks came from.

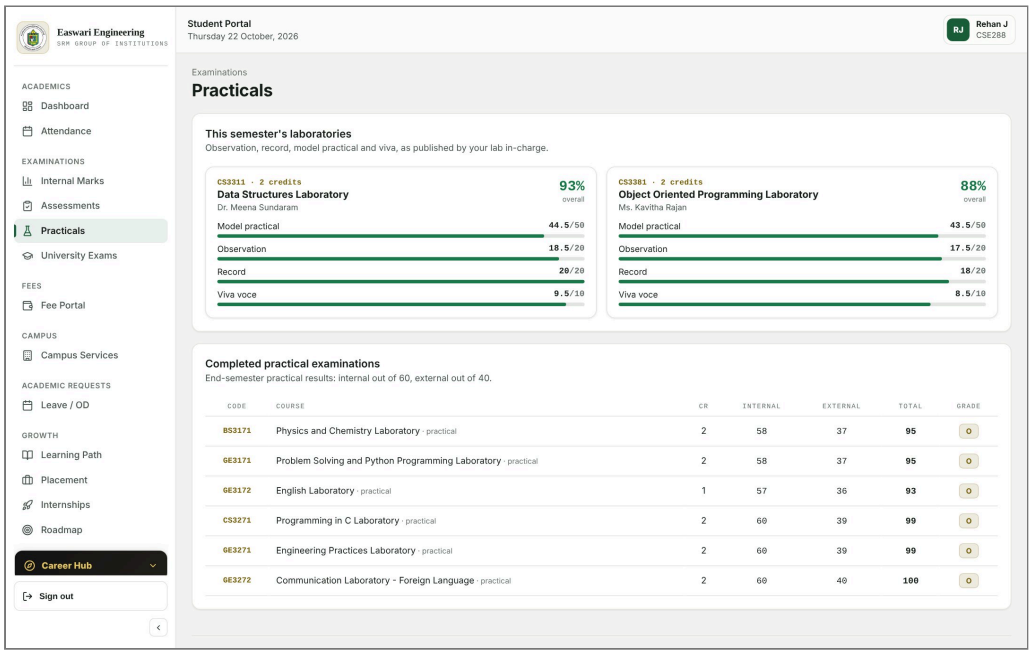


Figure 7.23: Practicals

7.24 UNIVERSITY EXAMINATION RESULTS

University Exams shows the cumulative grade point average, the credits earned, the number of standing arrears and the latest semester's average, and then lists each semester's papers with the course code, credits, internal and external marks, total and grade. The grade scale used is the university's ten-point scale.

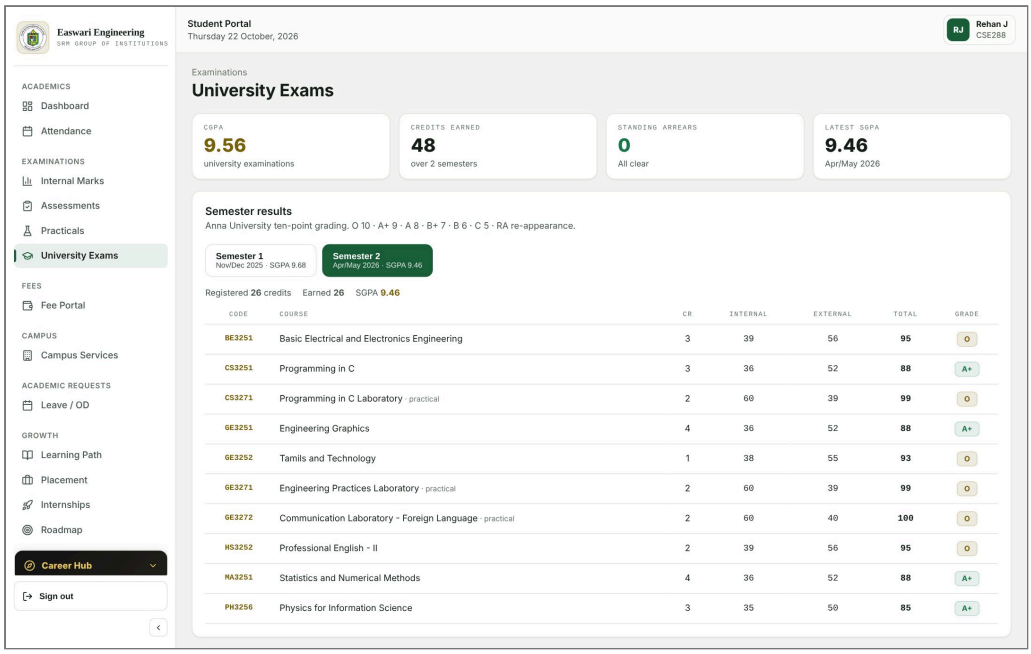


Figure 7.24: University examination results

7.25 FEE PORTAL

The Fee Portal shows the total billed, the amount paid, the amount due and the amount overdue. The Pay fees panel lists open items, with overdue items highlighted, and a student selects items and pays; a note states that payment here is simulated. The statement below lists every fee by academic year with its category, due date, amount, status and receipt number. Figures in this screenshot were taken after the payment tests of Chapter 6.

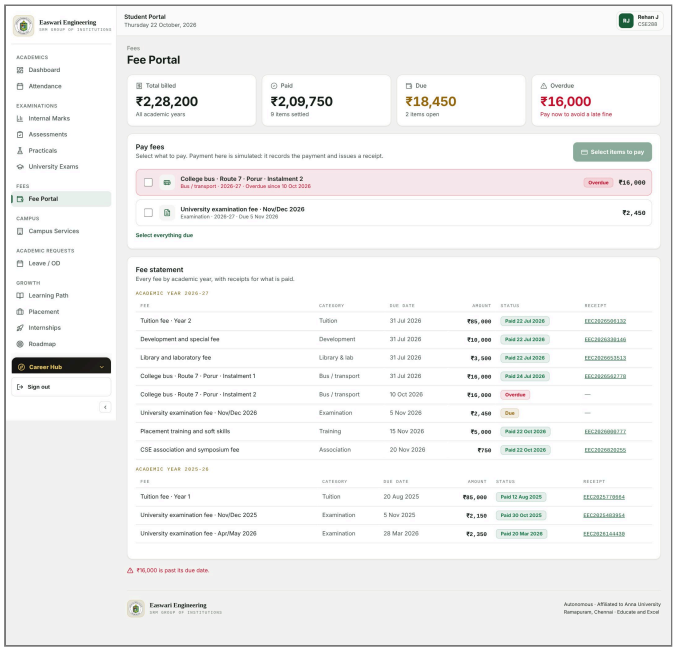


Figure 7.25: Fee portal

7.26 CAMPUS SERVICES

Campus Services collects the administrative requests that a student would otherwise make in person: fee payment, hostel services, transport, ID card, examination cell and library. Each card opens a request form; the list below shows the student's tickets with their status. The administrator sees the same tickets in the service queue.

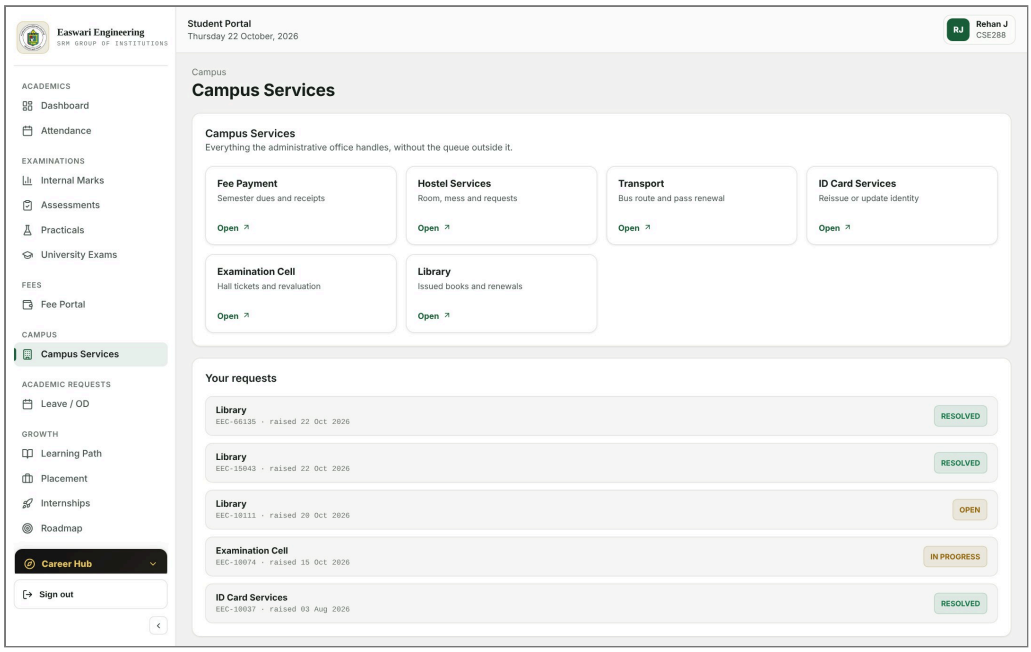


Figure 7.26: Campus services

7.27 LEAVE AND ON-DUTY APPLICATION

The student chooses the dates and the type, writes the reason and may attach a proof file. The application appears in the My requests list below with the status Pending, Approved or Rejected. The dates are validated, so a to-date earlier than the from-date is refused (test case TC39).

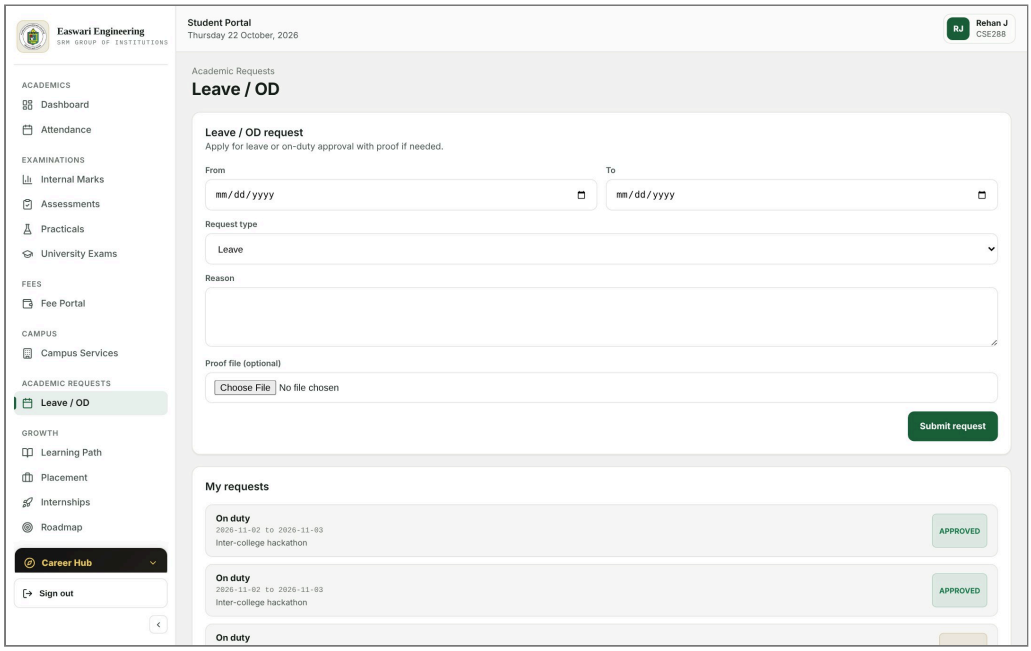


Figure 7.27: Leave and on-duty application

7.28 LEARNING PATH

The learning path orders recommendations by how much each would improve the student's standing and states the reason for each one, for example that communication is the lowest placement dimension. The recommendations are produced by the scoring service from the student's marks and placement scores. If the service is switched off the page shows a message instead of an error.

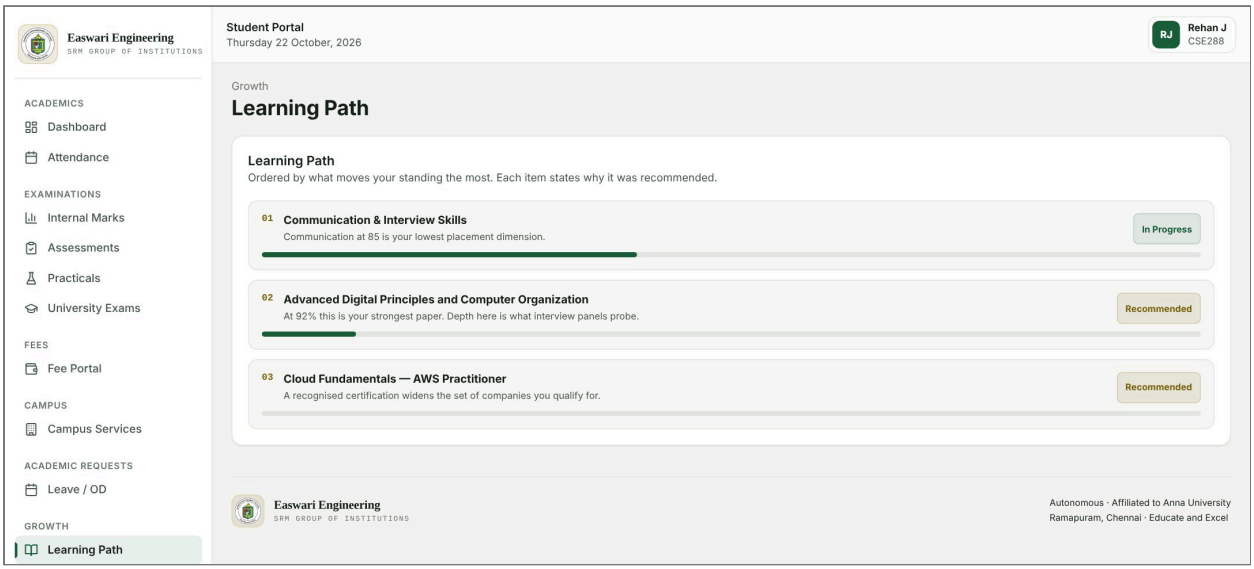


Figure 7.28: Learning path

7.29 PLACEMENT

The Placement page lists the companies that match the student's profile with a match percentage, the role, the package, the cut-off conditions on CGPA and attendance, and the closing date. A company whose criteria the student does not meet is marked Not eligible with the reason. The Apply button works only for eligible drives, and the server repeats the check (test case TC46).

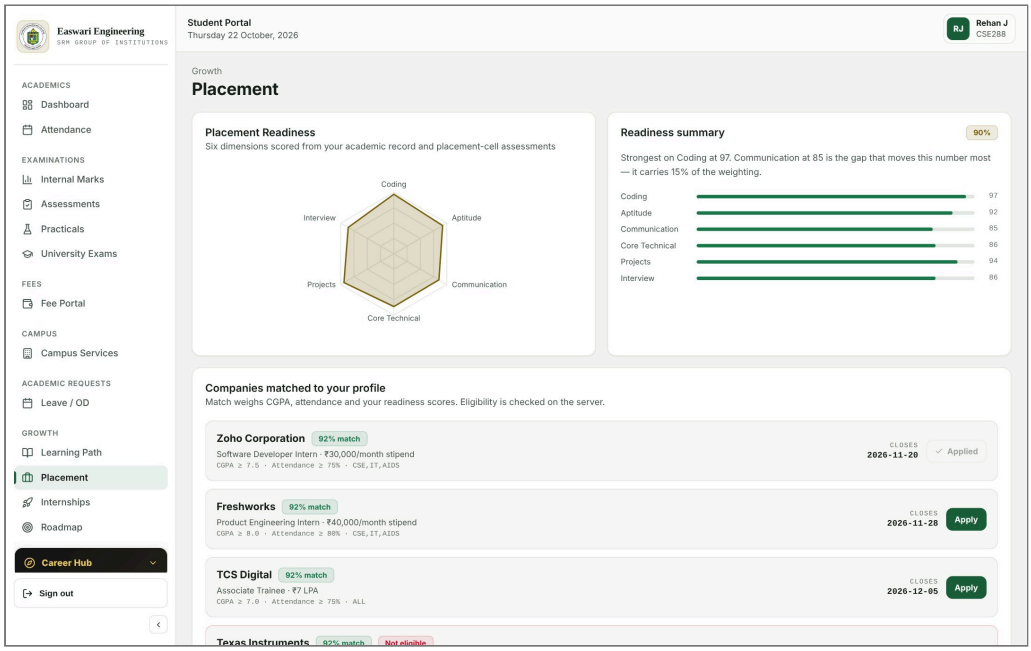


Figure 7.29: Placement

7.30 INTERNSHIPS

Internships shows the openings that are filtered for the student's branch, with the organisation, the duration, the stipend, the location and the days left to apply. A student can apply from the list.

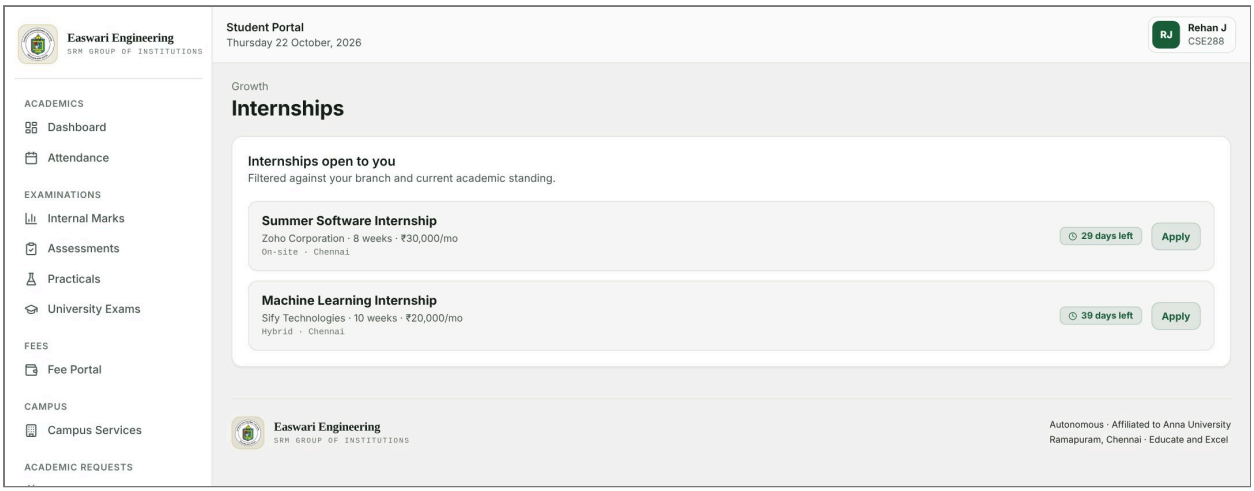


Figure 7.30: Internships

7.31 ROADMAP

The roadmap divides the programme into years and marks each as completed, in progress or upcoming, with the focus of the year: foundations, specialisation, core depth and placement season. The number of years is derived from the programme's duration field, so a programme of a different length needs no code change.

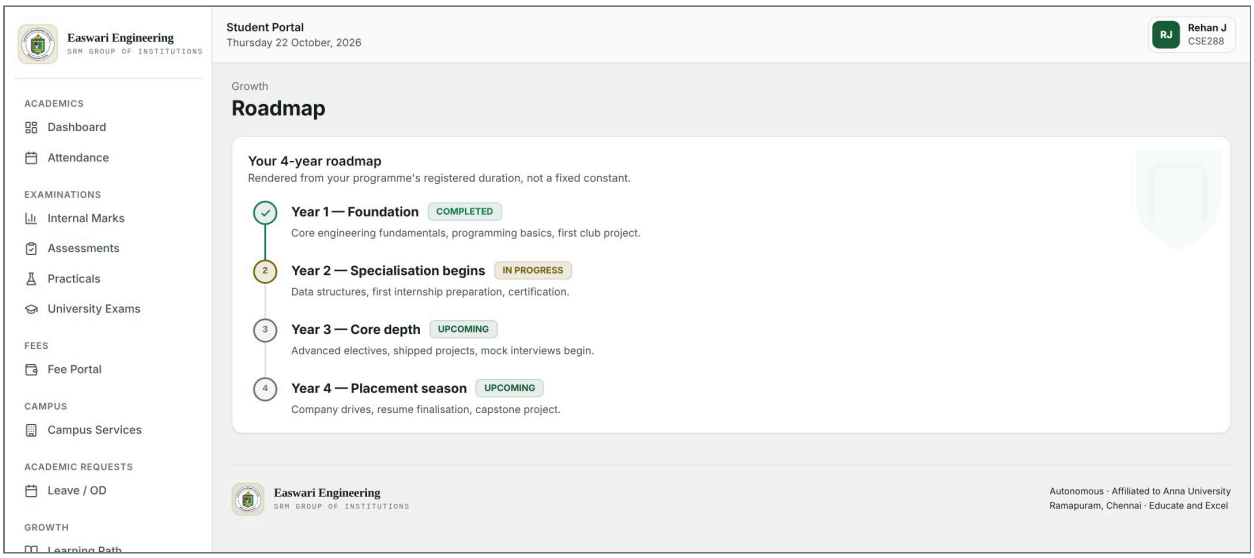


Figure 7.31: Roadmap

7.32 SETTINGS AND QUICK CODE

Settings lets the student create or remove the six-digit quick code. The student first confirms the current password and then enters the code in six slots. The same page is where a forgotten code is replaced; the code can never be set by anyone else.

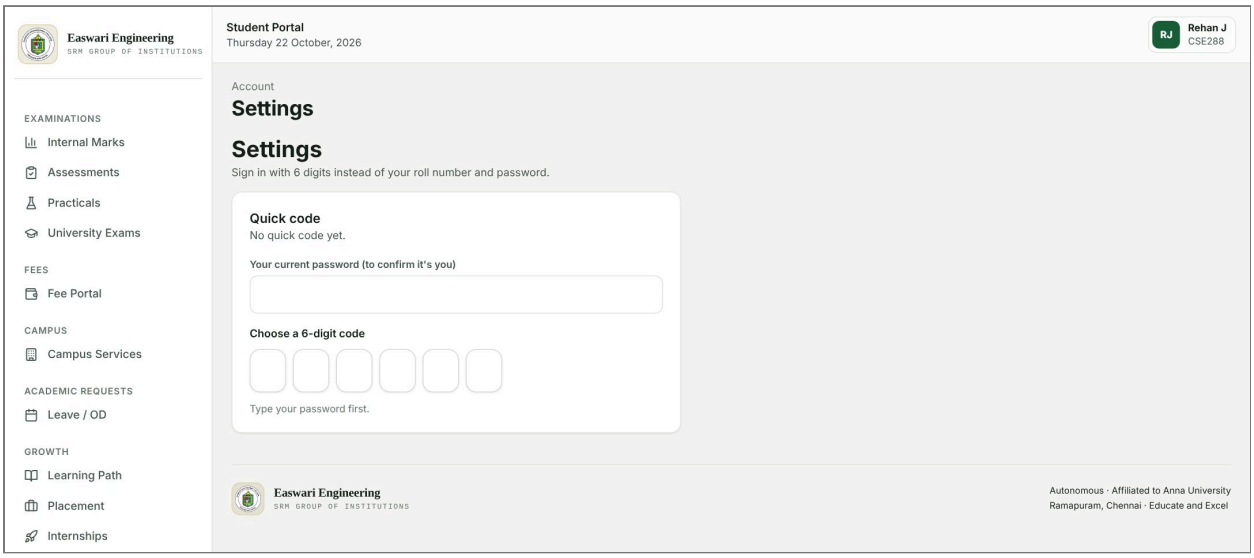


Figure 7.32: Settings and quick code

7.33 CAREER HUB OVERVIEW

The Career Hub is a boxed section of the sidebar and has its own black-and-gold theme. The overview greets the student with a readiness ring, a five-step path to placement and cards for each tool. Progress is stored on the student's own device and is kept separately for each student.

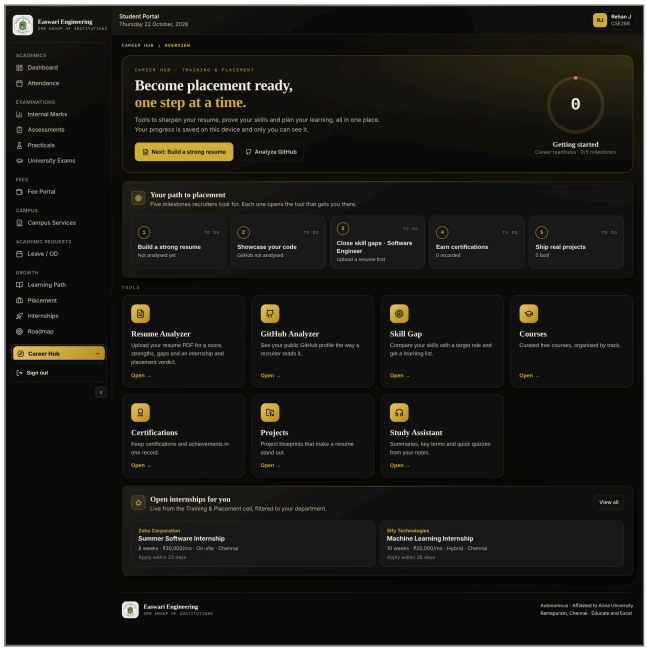


Figure 7.33: Career Hub overview

7.34 RESUME ANALYZER

The student chooses the target role and drops a resume in PDF form. The file is read in the browser with pdf.js and is not uploaded to any server. The analyzer reports a score, strengths, weaknesses, an upgrade plan and internship and placement readiness. Before a resume is loaded the page shows an instruction instead of an empty result.

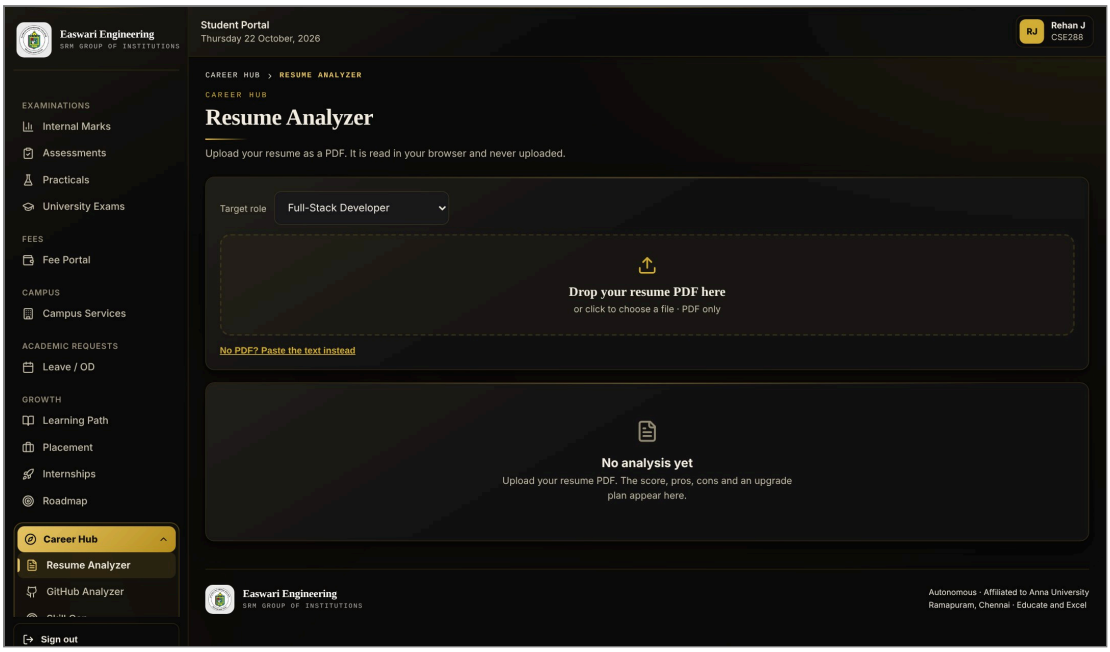


Figure 7.34: Resume analyzer

7.35 GITHUB ANALYZER

The GitHub analyzer reads a public profile and its repositories and scores how a recruiter would see them. The student enters a GitHub username and presses Analyze GitHub. Only public information is read.

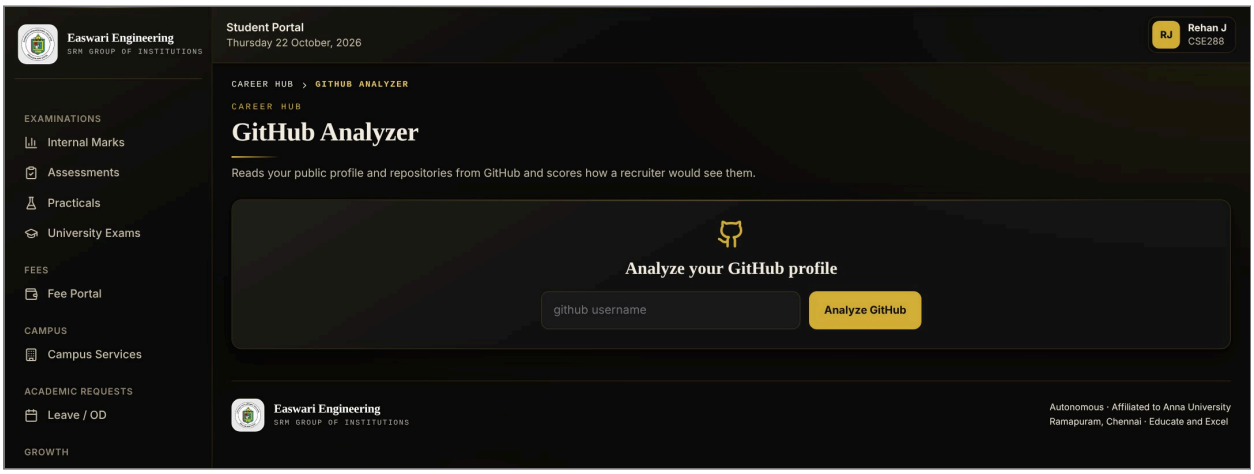


Figure 7.35: GitHub analyzer

7.36 SKILL GAP ANALYSIS

Skill Gap compares the skills found in the student's resume with those expected for a chosen target role, such as Full-Stack Developer, and lists what is missing. Until a resume has been analysed the page asks the student to open the Resume Analyzer first.

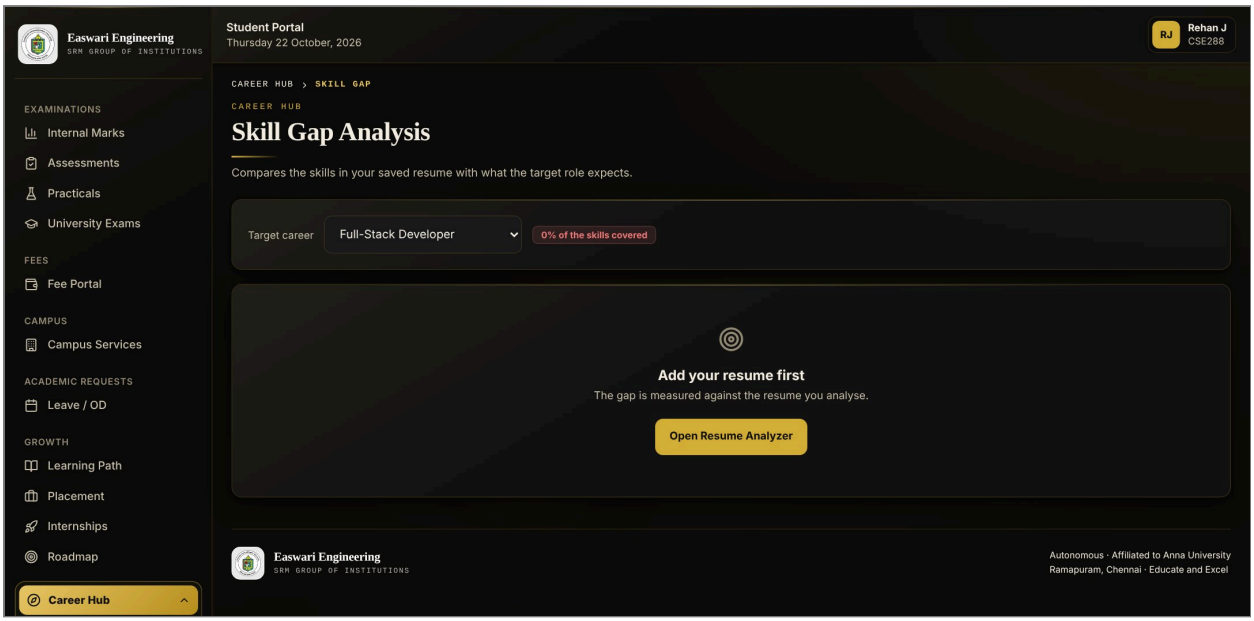


Figure 7.36: Skill gap analysis

7.37 COURSES

The Courses page lists free courses from well-known providers, grouped by topic and filterable by certificate, level and category. A student ticks a course as completed, and the overall course completion at the top is updated.

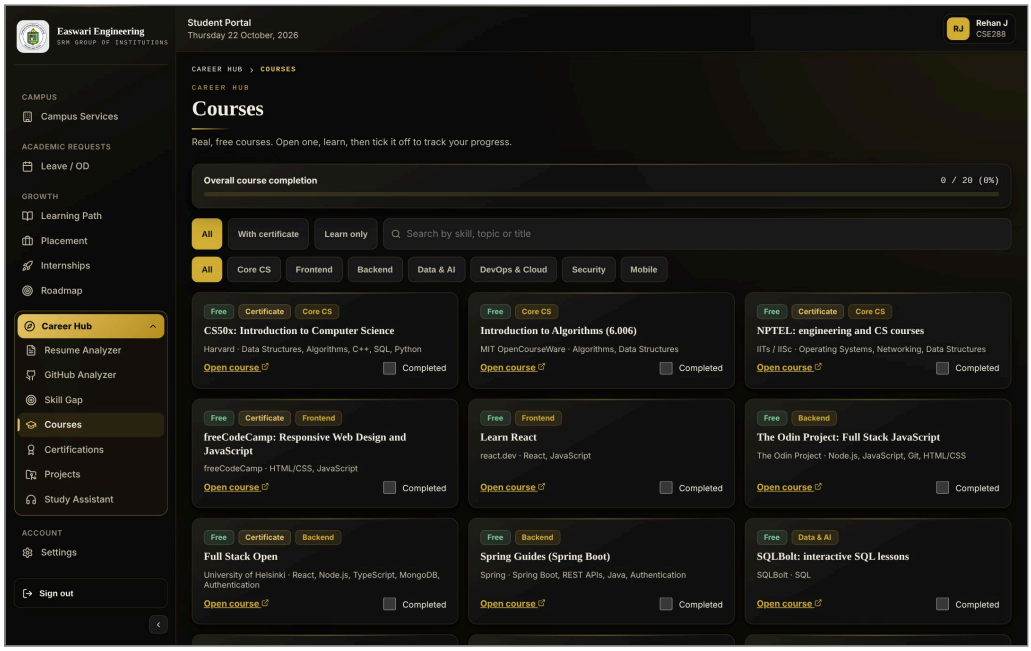


Figure 7.37: Courses

7.38 CERTIFICATIONS AND ACHIEVEMENTS

Students record certifications with the name, the issuer and an optional document. The list keeps the entries on the device and shows an empty state with a prompt when nothing has been added.

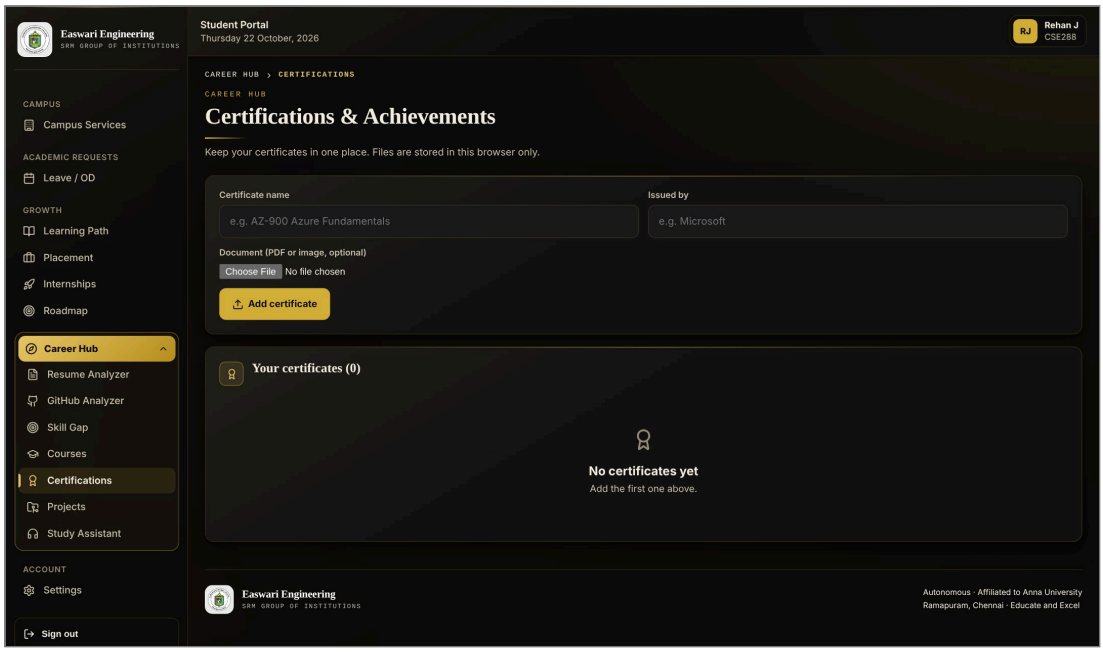


Figure 7.38: Certifications and achievements

7.39 PROJECT BLUEPRINTS

Project Blueprints suggests portfolio projects for the chosen role, with the difficulty, the features that a hiring manager looks for and the technology to use, and a field to record the repository link and a mark for built projects.

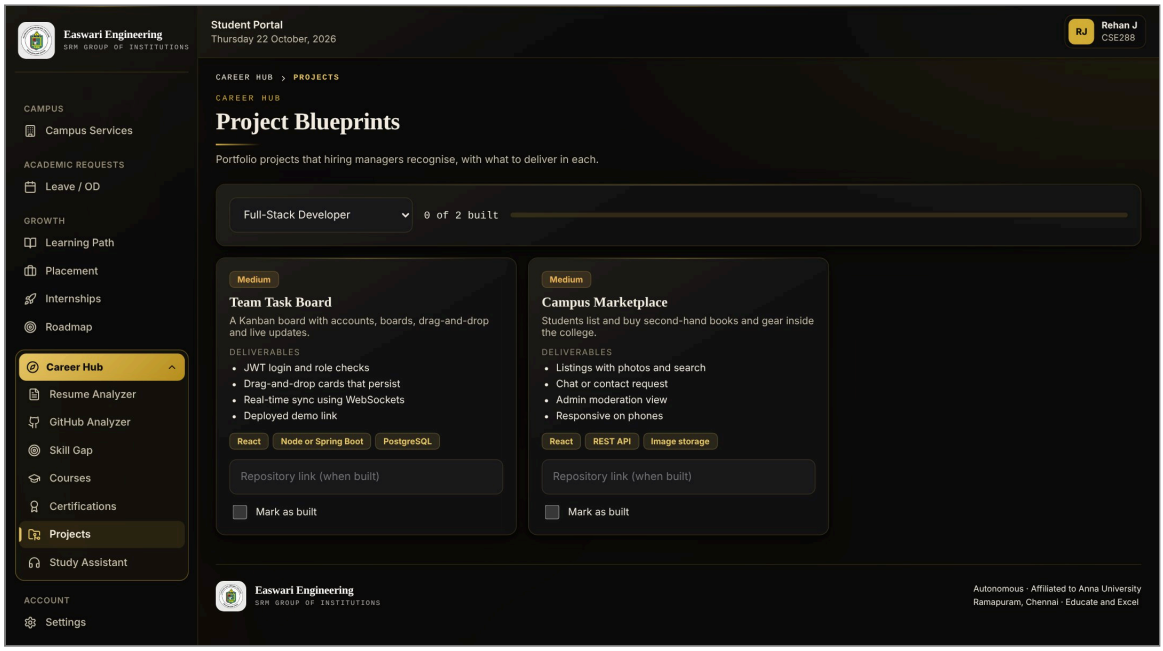


Figure 7.39: Project blueprints

7.40 STUDY ASSISTANT

The study assistant accepts notes or a text file and turns them into a summary, key terms and a practice quiz. The student gives the material a title, pastes the text and saves it; the saved materials are listed on the right.

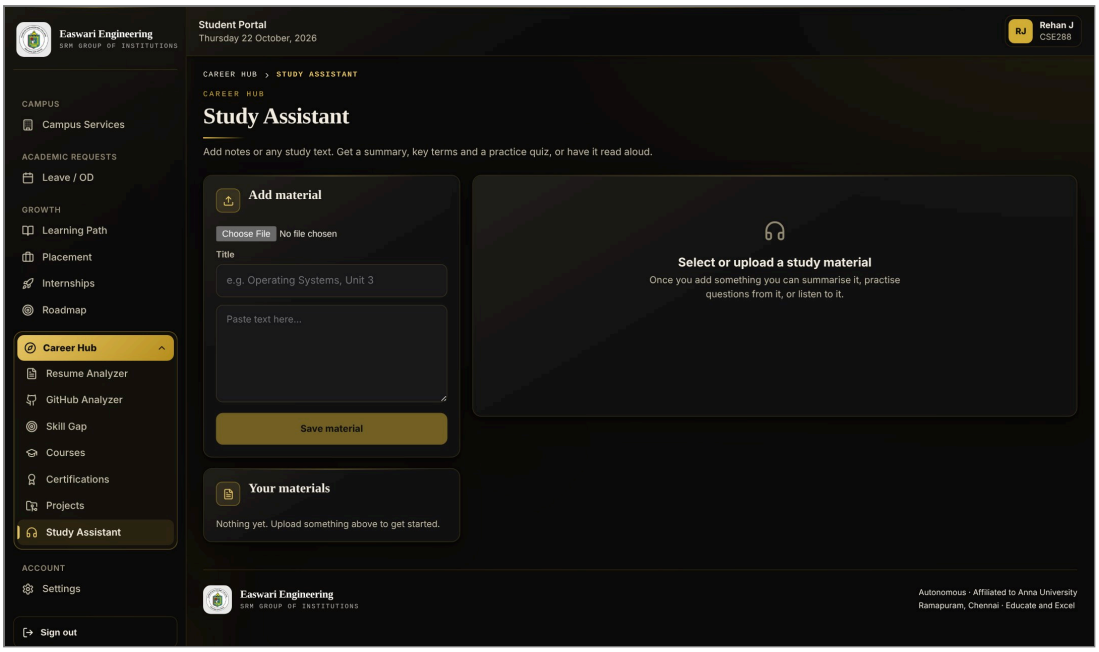


Figure 7.40: Study assistant

CHAPTER 8 – RESULTS AND DISCUSSIONS

8.1 RESULTS

The project produced a working three-portal platform that was started from a single packaged server file and a development front end, seeded with a complete institution and then exercised end to end. The packaged server started in about nine seconds on an ordinary laptop and created the full institution of five departments, ten classes, 59 students, 25 staff members and 28,497 attendance records in about ten seconds. Sixty-eight recorded test cases were run against the system; sixty-seven behaved as expected and one revealed a defect in the handling of an invalid status value. All eleven unit tests passed.

The central claim of the design, that the authority to change attendance and marks is enforced by the server, was confirmed by test cases TC17 to TC20, TC28, TC29 and TC54: a coordinator can mark and enter marks for every subject of the class, a subject teacher is refused for any other subject with a 403 response, and a teacher who has no connection with the class cannot even read its roster. The unique attendance slot made the repeated marking of a period an update, so the student’s percentage did not change. The leave workflow, the fee payment and the placement eligibility rules each refused the invalid case with a clear message.

8.2 DISCUSSION

Three decisions shaped the results. The first was to store attendance at the finest grain and derive everything else from it: the register grid, the subject table and the percentage on the dashboard are different views of the same rows and therefore cannot disagree. The second was to enforce permissions in the services and to take the identity of the student from the token; the unauthorized-access tests show that a changed address or a missing button does not matter. The third was to keep the scoring service outside the data layer; the tests in Table 6.17 show that the portals behave sensibly both with and without it.

The interface decisions are less measurable but were deliberate: a plain, neutral palette for the academic portals, solid cards instead of decorative effects, explicit page titles, and a separate black-and-gold theme only inside the Career Hub so that the placement tools feel different from the academic record.

8.3 LIMITATIONS

- Payment is simulated. No payment gateway is connected and no card or bank detail is collected.
- Defect D1: an unknown status value sent to the service-request update route returns a generic 500 response instead of a 400 message.
- Defect D2: the administrator’s password reset issues a role default value that must be changed at the next sign-in; it should use the random generator used for new accounts.

- The signing secret has a development default and the H2 console route is permitted; both must be changed before any production use.
- Lockout applies to the quick code only; repeated wrong passwords are refused but not yet rate limited.
- The scoring service is a transparent weighted heuristic and not a trained model; its weights need calibration with real outcomes.
- The resume and GitHub analyzers are rule-based, and Career Hub progress is stored on the device and is not shared between devices.
- The data shown is a seeded trial data set. University results, fees and attendance were generated for the trial and not imported from the college’s records.
- Layout on phone widths, the expiry of the fifteen-minute lock and long-running load were not tested.

8.4 EVALUATION METRICS TO RECORD

Table 8.1: Evaluation metrics recorded and to be recorded

Metric	Value / status
REST test cases executed	68 (67 passed, 1 defect)
Unit tests	11 of 11 passed
Server start-up time	About 9 seconds
Time to seed the full institution	About 10 seconds
Seeded records	5 departments, 10 classes, 59 students, 25 staff, 28,497 attendance records
Screens documented	40 screens across three portals
Executable jar size	About 72 MB
Time saved per period of attendance compared with a paper register	To be recorded in a pilot with faculty
Students who corrected short attendance after an early alert	To be recorded over a full semester
User satisfaction of faculty and students	To be recorded through a questionnaire

CHAPTER 9 – CONCLUSION AND FUTURE ENHANCEMENT

9.1 CONCLUSION

The Easwari ERP Portal demonstrates that attendance control and student progress monitoring can be handled in one role-based platform in which each fact is stored once and each rule is enforced where it cannot be bypassed. The platform gives students a single place to see their attendance, marks, results, fees and requests; gives faculty a quick way to mark and publish; gives the class coordinator authority over the whole class; and gives the administrator oversight and an audit trail.

The tests recorded in this report show that the rules claimed in the design hold in the running system, and the screens in Chapter 7 show that the information reaches the person who needs it. The limitations are stated plainly: the data is a trial data set, payment is simulated and a few hardening steps remain before real use. With those steps the platform is a sound foundation for a college deployment.

9.2 FUTURE ENHANCEMENTS

- Replace the simulated payment with a payment gateway in test mode and then in production, with webhook confirmation.
- Import real timetables, student lists and university results from the college’s own sources instead of the seeder.
- Notify students by e-mail or phone when attendance in a subject falls near 75% and notify parents on request.
- Add rate limiting for password sign-in and optional two-factor authentication for staff and administrators.
- Calibrate the risk weights against two or three semesters of real outcomes and add trend charts to the mentor view.
- Capture attendance by QR code or by a mobile application and reconcile it with the faculty entry, as discussed in the related work.
- Provide a mobile layout and an installable progressive web application.
- Synchronise Career Hub progress to the server so that a student sees the same state on every device.
- Add parent and mentor portals, a hostel module and a library module to the same sign-in.

9.3 RECOMMENDATIONS FOR FUTURE DEVELOPERS

1. Keep the attendance unique constraint and the ownership checks together; changing one without the other reintroduces the problems the design was made to remove.
2. Do not read the student’s identity from a path parameter. Resolve it from the token.

3. Fix defect D1 by validating the status value in the service-request controller and add a test case for it.
4. Replace the fixed reset password with the random generator and add a unit test beside the existing generator test.
5. Move the signing secret, the database URL and the scoring service address to environment variables and disable the H2 console route for production.
6. Run the unit tests and the REST checks in this report after every change to a service class.
7. Back up the database file before every upgrade and rehearse a restore as in Table 6.18.
8. Keep the scoring weights in one place and document every change to them.

CHAPTER 10 – APPENDIX

10.1 ATTENDANCE UNIQUENESS CONSTRAINT

The constraint below is the reason that marking a period twice cannot corrupt a percentage. The service looks up the existing row for the four-column key and updates it, and the database refuses a duplicate if two requests race.

Listing 10.1 – Constraint and indexes on attendance_records

```
@Table(name = "attendance_records",
    uniqueConstraints = @UniqueConstraint(
        name = "uk_attendance_slot",
        columnNames = {"student_id", "subject_offering_id",
                       "class_date", "period"}),
    indexes = {
        @Index(name = "idx_att_student_date",
               columnList = "student_id, class_date"),
        @Index(name = "idx_att_offering_date",
               columnList = "subject_offering_id, class_date")
    })
```

10.2 ROLE AUTHORIZATION PATTERN

Three layers combine. The route guard decides on the address prefix; the controller methods carry role annotations; the service checks ownership. A request must pass all three.

Listing 10.2 – The three layers of authorization

```
// 1. route guard (SecurityConfig)
.requestMatchers("/api/faculty/**").hasAnyRole("FACULTY", "ADMIN")

// 2. controller (class-level role check)
@RestController @RequestMapping("/api/faculty")
@PreAuthorize("hasAnyRole('FACULTY','ADMIN')")

// 3. service (ownership)
if (!teaches && !coordinates)
    throw new ResponseStatusException(HttpStatus.FORBIDDEN,
            "You are not assigned to this class");
```

10.3 RISK SCORE COMPARISON

The table gives the verified behaviour of the risk model for three student profiles, as documented with the scoring service. It shows that the score rises steeply when attendance falls below the 75% line.

Table 10.1: Risk score for three profiles

Profile	Attendance	CGPA	Score	Band
Healthy	93%	8.90	0	LOW
Borderline	78%	7.20	36	MEDIUM
Struggling	71%	6.40	66	HIGH

10.4 KEY VIVA QUESTIONS

Q. Why is attendance stored per period and not per day?
A. Because period-level rows are the finest grain. The per-day and per-subject views are aggregations of the same rows, so they can never disagree.

Q. What stops a period from being marked twice?
A. The unique constraint uk_attendance_slot on student, offering, date and period; the service updates the existing row.

Q. Who may change attendance for a class?
A. The administrator, the coordinator of the class for every subject, and a teacher for his or her own subject only.

Q. Why is the student's identity taken from the token?
A. So that changing a number in the address cannot expose another student's record.

Q. Why a separate scoring service?
A. Java owns the data and Python owns the models. The boundary lets each be scaled or replaced independently and lets the portals work when the scoring service is off.

Q. Is the risk score a prediction?
A. No. It is a transparent weighted score with listed factors; the weights are to be calibrated with real outcomes.

Q. Why H2 and how does the system move to PostgreSQL?
A. H2 in file mode needs no installation yet enforces constraints. A production profile changes only the connection settings.

Q. What happens to a student's marks if a teacher saves a blank field?
A. The stored mark is unchanged; a blank means not entered, which is different from zero.

Q. How is a temporary password protected?
A. It is random, shown once, hashed in the database and must be changed at the first sign-in.

Q. What are the known limitations?
A. Simulated payment, a trial data set, defects D1 and D2, development defaults for the secret and the H2 console.

CHAPTER 11 – REFERENCES

[1] F. Ramadhani and A. Satria, “Implementation of a QR Code-Based Attendance System for Students Using Web Technology,” *Altafani: Jurnal Pengabdian Masyarakat*, vol. 2, no. 1, pp. 12–18, Mar. 2025.

[2] G. Akçapınar, A. Altun and P. Aşkar, “Using learning analytics to develop early-warning system for at-risk students,” *International Journal of Educational Technology in Higher Education*, vol. 16, no. 1, pp. 1–20, Oct. 2019, doi: 10.1186/s41239-019-0172-z.

[3] E. Howard, M. Meehan and A. Parnell, “Contrasting Prediction Methods for Early Warning Systems at Undergraduate Level,” arXiv:1612.05735, 2017.

[4] M. Jones, J. Bradley and N. Sakimura, “JSON Web Token (JWT),” RFC 7519, Internet Engineering Task Force, May 2015.

[5] N. Provos and D. Mazières, “A Future-Adaptable Password Scheme,” in *Proc. USENIX Annual Technical Conference*, 1999.

[6] R. T. Fielding, “Architectural Styles and the Design of Network-based Software Architectures,” Ph.D. dissertation, University of California, Irvine, 2000.

[7] VMware, Inc., *Spring Boot Reference Documentation*, version 3.2.x. [Online]. Available: https://docs.spring.io/spring-boot/

[8] Meta Platforms, Inc., *React Documentation*, version 18. [Online]. Available: https://react.dev

[9] Vite contributors, *Vite Guide*. [Online]. Available: https://vitejs.dev

[10] S. Ramírez, *FastAPI Documentation*. [Online]. Available: https://fastapi.tiangolo.com

[11] H2 Database Engine, *H2 Database Documentation*. [Online]. Available: https://h2database.com

[12] The Apache Software Foundation, *Apache POI – Java API for Microsoft Documents*. [Online]. Available: https://poi.apache.org

[13] Mozilla Foundation, *PDF.js*. [Online]. Available: https://mozilla.github.io/pdf.js/

[14] OWASP Foundation, *Application Security Verification Standard (ASVS)*, version 4.0.3, 2021. [Online]. Available: https://owasp.org/www-project-application-security-verification-standard/